BIBLIOTECA IA, SOFTWARE & COMPUTAÇÃO · LIVRO 1

Fundamentos de IA e Modelos de Linguagem

Do que é inteligência artificial e aprendizado de máquina até como funcionam os grandes modelos de linguagem, Transformers, Mistura de Especialistas e LoRA.

Edição de outubro de 2026

Sobre esta coleção

> **Para começar**
> Esta coleção explica inteligência artificial, desenvolvimento de software e computação do zero. Cada capítulo começa em linguagem simples, com analogias do dia a dia, e aprofunda aos poucos. Os livros podem ser lidos separadamente.

O conteúdo reorganiza o que ensinam centenas de aulas e vídeos (do CS50 de Harvard a canais como IBM Technology, Anthropic e muitos outros), artigos técnicos de referência e o blog AkitaOnRails, complementados com pesquisa atualizada. Os vídeos aparecem pelo título, com link; cada capítulo termina com a tabela de fontes. Caixas *Você sabia?* trazem curiosidades e grandes nomes da tecnologia.

Os livros da coleção

1. Livro 1 — Fundamentos de IA e Modelos de Linguagem
2. Livro 2 — Construindo com LLMs
3. Livro 3 — Programando com IA
4. Livro 4 — Engenharia de Software e Ciência da Computação
5. Livro 5 — IA, Negócios e Sociedade
6. Livro 6 — Cultura, Comunidade e História

> **Datas**
> IA muda depressa. Preços, planos, benchmarks e nomes de modelos refletem outubro de 2026. Figuras marcadas como "ilustrativas" mostram a ideia, não medições.

Sumário

1. Fundamentos de IA, Machine Learning e Deep Learning 4
2. LLMs, Transformers e Mistura de Especialistas 22

CAPÍTULO 1 ·

Fundamentos de IA, Machine Learning e Deep Learning

> *Antes dos agentes, dos LLMs e dos modelos de difusão existe uma base que não envelhece: formular problemas como busca, raciocinar com lógica e probabilidade, otimizar, aprender com dados e empilhar neurônios artificiais em redes profundas. Este capítulo percorre esse caminho em ordem, do algoritmo de busca em largura ao Q-learning rodando no navegador, apoiado no curso CS50 AI de Harvard, em vídeos do canal IBM Technology e em textos técnicos da DataCamp, da Notable Capital, do CNN Explainer e de um tutorial de TensorFlow.js.*

Para começar

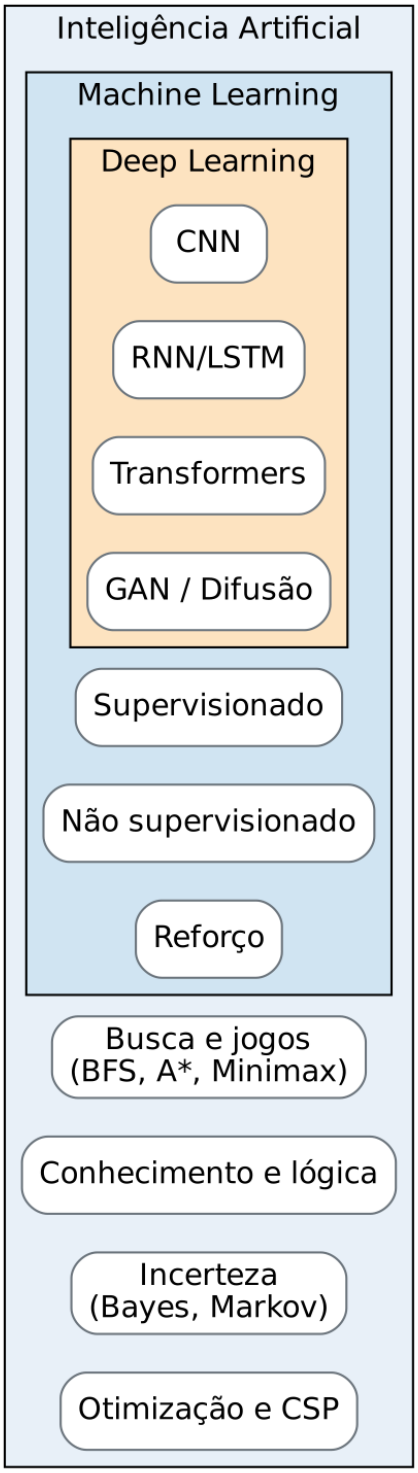
Ninguém aprende a andar de bicicleta com uma lista de regras: você tenta, cai, ajusta e, depois de muitos exemplos, o corpo "aprende". A IA moderna funciona assim: em vez de escrever todas as regras, mostramos exemplos e o computador se ajusta. Mas antes disso a IA passou por ideias mais simples (procurar caminhos, deduzir, calcular probabilidades) que continuam dentro dos sistemas atuais.

O que é (e o que não é) inteligência artificial

Para começar

Pense em bonecas russas: a maior é a IA (qualquer técnica que faça o computador agir de modo "esperto"); dentro dela está o aprendizado de máquina (técnicas que aprendem com exemplos); dentro desta, o aprendizado profundo (redes neurais com muitas camadas).

Inteligência artificial é o campo que constrói sistemas capazes de tarefas que, em humanos, exigiriam inteligência: entender linguagem, reconhecer padrões, planejar, decidir. Um jogo da velha que nunca perde é IA; um filtro de spam também; um LLM também. O que muda é *como* o comportamento é obtido. Regras escritas à mão ("se a bola está à esquerda, mova a raquete para a esquerda") funcionam em domínios minúsculos, mas não escalam. A história da IA é, em boa parte, a troca de regras explícitas por **busca** e depois por **aprendizado**.



IA contém ML, que contém DL. Busca, lógica, probabilidade e otimização são IA "clássica", sem necessariamente aprender de dados. Fonte: elaboração própria a partir de DataCamp (Crabtree, 2024) e CS50 AI.

- **IA**: o objetivo geral; inclui técnicas simbólicas sem aprendizado.
- **Machine Learning (ML)**: algoritmos que *aprendem com dados* em vez de serem programados explicitamente (árvores de decisão, Naive Bayes, SVM, k-NN, k-means).
- **Deep Learning (DL)**: ML com redes neurais de várias camadas, que aprendem representações hierárquicas (de bordas a partes a objetos).

IA preditiva e IA generativa

Outra divisão, do vídeo Predictive vs Generative AI: How They Work and When to Use Each, é pela pergunta respondida. A **preditiva** responde "o que vai acontecer?" com um número, classe ou probabilidade verificável ("87% de chance de este cliente cancelar"). A **generativa** responde "como isso poderia ser?" com texto, imagem ou código, sem resposta única correta.

Aspecto	IA preditiva	IA generativa
Saída	Valor, classe, probabilidade	Texto, imagem, áudio, código
Dados típicos	Estruturados (tabelas)	Não estruturados
Avaliação	Contra a verdade observada	Qualidade subjetiva, preferência humana
Algoritmos	Regressão, árvores, gradient boosting	Transformers, difusão, GANs

As duas se combinam: um modelo preditivo aponta clientes com risco de cancelar e um generativo redige o e-mail de retenção. Um LLM tecnicamente *prevê o próximo token*, mas seu propósito é generativo.

> **Você sabia?**
> O termo "inteligência artificial" nasceu numa proposta de 1955 de John McCarthy, Marvin Minsky, Nathaniel Rochester e Claude Shannon para um workshop em Dartmouth, realizado em 1956. Eles apostavam num "avanço significativo" em dois meses de trabalho de dez pessoas.

Busca

> **Para começar**
> Num labirinto, resolver o problema é *procurar*: experimentar caminhos, descartar becos sem saída e escolher o melhor. Rotas de GPS, quebra-cabeças e xadrez podem ser descritos assim.

A primeira aula do curso Harvard CS50's Artificial Intelligence with Python – Full University Course define um **problema de busca** por: estado inicial; **ações** possíveis em cada estado; **modelo de transição** (o estado resultante de uma ação); **teste de objetivo**; e **custo de caminho**. Estados e transições formam um grafo, e a **solução ótima** é o caminho de menor custo até o objetivo.

```
fronteira ← {nó(estado_inicial)}
explorados ← ∅
repita:
    se fronteira vazia: retorne FALHA
    n ← fronteira.remover()
    se objetivo(n.estado): retorne caminho(n)
    explorados.add(n.estado)
    para cada a em ACTIONS(n.estado):
        s' ← RESULT(n.estado, a)
        se s' ∉ explorados e s' ∉ fronteira:
            fronteira.add(nó(s', pai=n, ação=a))
```

O conjunto de explorados evita laços infinitos. A diferença entre os algoritmos clássicos está só na estrutura da fronteira:

Algoritmo	Fronteira	Ótimo?	Observação
DFS (profundidade)	Pilha (LIFO)	Não	Pouca memória; pode achar caminho longo
BFS (largura)	Fila (FIFO)	Sim, com custos unitários	Memória cresce exponencialmente
Custo uniforme (Dijkstra)	Prioridade por g(n)	Sim	BFS com custos variáveis
Greedy best-first	Prioridade por h(n)	Não	Rápida, mas a heurística engana
A*	Prioridade por g(n) +h(n)	Sim, se h admissível	Padrão em rotas e jogos

Busca informada e heurísticas

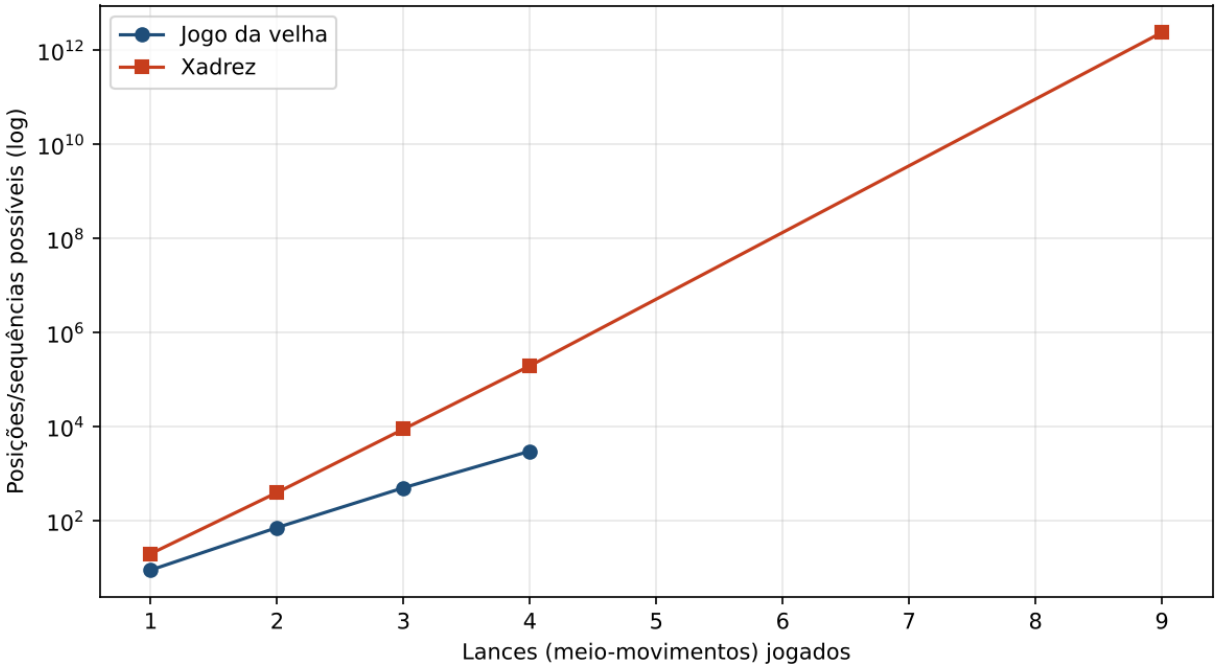
Busca *informada* usa uma **heurística** h(n), estimativa do custo restante; num labirinto em grade, a distância de Manhattan |Δx|+|Δy|. A greedy segue só h(n) e cai em corredores que terminam numa parede; o **A*** soma o custo já pago, g(n). Ele é ótimo se h for **admissível** (nunca superestima o custo real). Na prática, a qualidade do A* é a qualidade da heurística, e heurísticas obtidas "relaxando" o problema (ignorar paredes) costumam ser admissíveis.

Busca adversarial: Minimax e alfa-beta

Em jogos há um adversário. O **Minimax** atribui utilidade aos estados finais (no jogo da velha: X vence = +1, empate = 0, O vence = −1); MAX escolhe a ação de maior valor, MIN a de menor, recursivamente:

```
def valor(s):
    if terminal(s): return utilidade(s)
    if jogador(s) == MAX:
        return max(valor(result(s, a)) for a in acoes(s))
    return min(valor(result(s, a)) for a in acoes(s))
```

A **poda alfa-beta** abandona ramos que não podem mudar a decisão e, com boa ordenação, quase dobra a profundidade alcançável. O obstáculo é a **explosão combinatória**: o jogo da velha tem 255.168 partidas possíveis; o xadrez, segundo a palestra CS50 for Business - Lecture 4 - Approaching Artificial Intelligence, cerca de 2,4 trilhões de sequências após 9 meio-movimentos.



Crescimento do número de sequências de jogo (escala log): jogo da velha e xadrez. Fonte: elaboração própria com dados de "CS50 for Business - Lecture 4 - Approaching Artificial Intelligence" e contagens "perft" do xadrez.

A saída é limitar a profundidade e usar uma **função de avaliação** (por exemplo, o saldo de material no xadrez). Foi aí que o aprendizado entrou nos jogos: o AlphaGo *aprende* a avaliação e a política de jogadas e as combina com busca em árvore Monte Carlo.

Conhecimento e lógica

> **Para começar**
> Um detetive junta pistas e deduz conclusões que *têm* de ser verdadeiras. A lógica dá ao computador essa capacidade: escrevemos fatos e regras, e ele tira conclusões garantidas.

Na **lógica proposicional** há símbolos (P = "está chovendo"), conectivos (¬, ∧, ∨, →, ↔) e **modelos**, atribuições de verdadeiro/falso a cada símbolo. Uma **base de conhecimento** (KB) **acarreta** α (KB ⊨ α) se α é verdadeira em todo modelo em que a KB é verdadeira.

- **Model checking**: enumerar os 2^n modelos e verificar. Correto e completo, mas exponencial. O CS50 resolve com ele o jogo Detetive (Clue).
- **Regras de inferência** (Modus Ponens, De Morgan etc.): encontrar uma prova vira, de novo, um problema de busca.
- **Resolução**: com a KB em forma normal conjuntiva, acrescenta-se ¬α e combinam-se cláusulas (P∨Q e ¬P∨R produzem Q∨R) até chegar à cláusula vazia, uma contradição. É a base dos provadores automáticos e, indiretamente, dos solucionadores SAT usados em verificação de hardware e software.

A **lógica de primeira ordem** acrescenta objetos, predicados e quantificadores (∀, ∃). Lógica dá certeza e explicabilidade, mas exige conhecimento escrito à mão e um mundo preto-no-branco.

Incerteza

> **Para começar**
> No mundo real quase nada é certo: "70% de chance de chuva", exames com falso positivo. Em vez de "verdadeiro ou falso", precisamos raciocinar com "quão provável".

O essencial: probabilidade condicional P(a | b) = P(a ∧ b) / P(b) e a **regra de Bayes**, P(b | a) = P(a | b)·P(b) / P(a), que inverte a direção do conhecimento: de "quão provável é a evidência dada a causa" para "quão provável é a causa dada a evidência". Exemplo clássico: um teste com 99% de sensibilidade, 5% de falsos positivos e doença com prevalência de 1%. Um positivo indica doença em apenas 0,99·0,01 / (0,99·0,01 + 0,05·0,99) ≈ 16,7%. É o mesmo raciocínio do filtro de spam Naive Bayes.

- **Redes bayesianas**: grafos acíclicos em que cada nó é uma variável e guarda P(X | pais). No exemplo do curso, chuva → manutenção da via → atraso do trem → comparecer à reunião. Quando a inferência exata é cara, usa-se amostragem.
- **Modelos de Markov**: o estado atual depende só do anterior. Um **modelo oculto de Markov** (HMM) separa estados escondidos (está chovendo?) de observações (a pessoa trouxe guarda-chuva?). HMMs dominaram o reconhecimento de fala por décadas.

Otimização

> **Para começar**
> Às vezes queremos a melhor *arrumação*, não um caminho: onde construir hospitais, como montar um horário de provas sem conflitos. É como procurar o ponto mais alto de uma região montanhosa na neblina, enxergando só ao redor.

- **Busca local**: o **hill climbing** anda sempre para o melhor vizinho e pode parar num máximo local. O **simulated annealing** aceita movimentos piores com probabilidade que cai ao longo do tempo: explora muito no início e pouco no fim.
- **Programação linear**: com objetivo e restrições lineares, o simplex encontra o ótimo global (o curso usa `scipy.optimize.linprog`).
- **Satisfação de restrições (CSP)**: variáveis, domínios e restrições, como agendar provas sem conflito ou resolver Sudoku. Usa consistência de arco (AC-3), backtracking e heurísticas como "variável com menos valores restantes".

A otimização é o elo com o aprendizado: treinar uma rede neural é minimizar uma função de perda, usando o *gradiente* em vez de vizinhos discretos.

Aprendizado de máquina: os três paradigmas

> **Para começar**
> No aprendizado de máquina entregamos exemplos e o computador descobre o padrão, como uma criança que aprende a reconhecer cachorros vendo muitos cachorros. Há três jeitos de ensinar: com gabarito, sem gabarito e por recompensa.

Paradigma	Sinal de treino	Exemplos	Algoritmos típicos
Supervisionado	Pares (entrada, rótulo)	Spam, preço de imóvel, diagnóstico	Regressão, k-NN, SVM, árvores, redes neurais
Não supervisionado	Só entradas	Agrupar clientes, reduzir dimensão	k-means, PCA/SVD, autoencoders
Por reforço	Recompensa após ações	Jogos, robótica, RLHF	Q-learning, PPO
Autossupervisionado	Rótulos derivados dos próprios dados	Pré-treino de LLMs (prever o próximo token)	Modelos de linguagem, contrastivo

Aprendizado supervisionado

Dados pares (x, y), buscamos uma função h que aproxime y = f(x): **classificação** se y é discreto, **regressão** se contínuo. Os clássicos são k-NN (voto dos vizinhos mais próximos), perceptron (fronteira linear), SVM (fronteira de máxima margem; com *kernels*, não linear), regressão logística e árvores/gradient boosting, que seguem como estado da arte em dados tabulares.

Overfitting é decorar o treino e generalizar mal. O vídeo What is Semi-Supervised Learning? dá um exemplo: se todas as fotos de gatos foram tiradas dentro de casa e as de cães ao ar livre, o modelo aprende "ao ar livre ⇒ cão". As defesas são regularização (penalizar complexidade, dropout), separar dados de teste nunca vistos (holdout, validação cruzada) e mais diversidade de dados.

Um caso clássico de ML supervisionado, citado pela DataCamp: 1.000 casas, a feature "número de quartos" e uma árvore de decisão que aprende a relação entre quartos e preço. O algoritmo traça uma **fronteira de decisão** a partir das features, e a qualidade só é medida de verdade em dados que ele nunca viu. Funções de perda formalizam o erro: perda 0-1 (acertou ou errou), L1 = $|y - \hat{y}|$, L2 = $(y - \hat{y})^2$ e entropia cruzada para probabilidades.

Avaliação: matriz de confusão

O tutorial Confusion to Clarity: Mastering Confusion Matrix in Machine Learning treina regressão logística no dataset de câncer de mama do scikit-learn e obtém acurácia de 95%, precisão de 94% e recall de 97%:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn import metrics

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25)
sc = StandardScaler(); X_tr = sc.fit_transform(X_tr); X_te = sc.transform(X_te)
y_pred = LogisticRegression().fit(X_tr, y_tr).predict(X_te)
print(metrics.confusion_matrix(y_te, y_pred))
```

Métrica	Fórmula	Pergunta que responde
Acurácia	(VP+VN)/total	Quanto acertei no geral?
Precisão	VP/(VP+FP)	Dos que marquei positivos, quantos eram?
Recall	VP/(VP+FN)	Dos positivos reais, quantos encontrei?
F1	2·P·R/(P+R)	Equilíbrio entre precisão e recall

Atenção

Em `load_breast_cancer`, a classe 1 é *benigno*. Com o padrão do scikit-learn, você mede a detecção de tumores benignos; calcule as métricas para a classe que importa. E acurácia engana em dados desbalanceados: um detector que diz "nunca é fraude" tem 99,9% de acurácia.

Não supervisionado, poucos rótulos e zero-shot

Sem rótulos, procuram-se estruturas. O **k-means** alterna atribuir cada ponto ao centróide mais próximo e mover o centróide para a média dos seus pontos. A **redução de dimensionalidade**, tema de How Linear Algebra Powers Machine Learning (ML), usa a **SVD** ($A = U\Sigma V^T$) para manter só as direções mais importantes; é a base do PCA e da ideia de atualizações de baixo posto do LoRA.

Rotular é caro. O aprendizado **semissupervisionado** combina poucos rótulos com muitos dados sem rótulo: pseudo-rotulagem (o modelo rotula e as previsões confiantes entram no treino), clustering e *active learning* (os casos duvidosos vão para humanos). No limite, o **zero-shot learning** (What is Zero-Shot Learning?) classifica classes nunca vistas, por exemplo pondo imagens e descrições no mesmo espaço vetorial, a ideia do CLIP. Entre os dois extremos ficam o *few-shot* e o *one-shot*, que aprendem uma classe nova a partir de poucos exemplos ou de um só, e a geração de dados sintéticos, outra forma de contornar a falta de rótulos.

Classificação de texto como estudo de caso

A classificação de texto, tema de Text Classification: AI Techniques and Real-World Applications, reúne quase tudo acima. Há três tipos: **binária** (spam ou não), **multiclasse** (pedido, suporte ou reclamação) e **multirrótulo** (um filme pode ser "ação" e "aventura"). O pipeline é iterativo: pré-processamento (a etapa mais demorada), extração de features por embeddings, modelo (de Naive Bayes a um LLM) e rótulos de saída. Os desafios recorrentes são desbalanceamento entre classes, ambiguidade ("manga" fruta ou de cami-

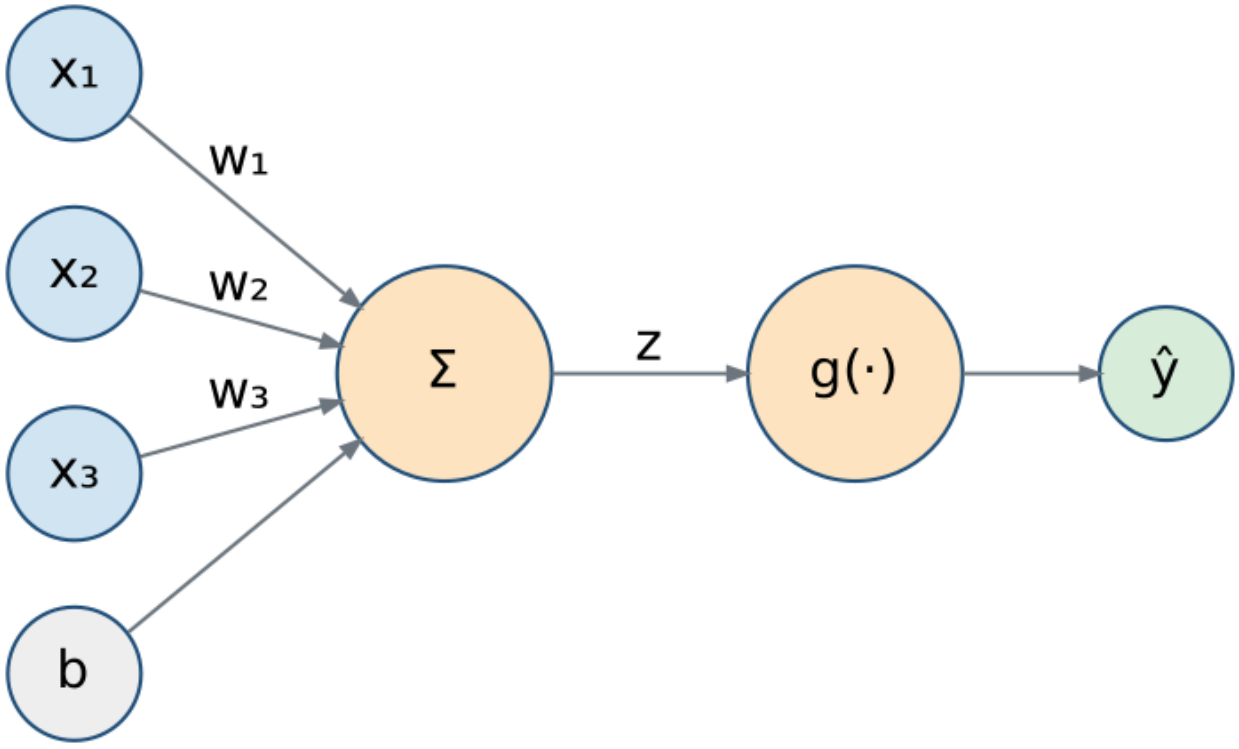
sa) e **drift**: o sentimento sobre um tema muda com os eventos do mundo, e o modelo precisa de validação contínua.

Viés algorítmico

O viés raramente nasce do algoritmo; nasce dos dados. O vídeo Algorithmic Bias in AI: What It Is and How to Fix It aponta dados históricos enviesados, amostras não representativas e variáveis-proxy (o CEP como proxy de etnia), e defende dados diversos, auditorias e governança contínua.

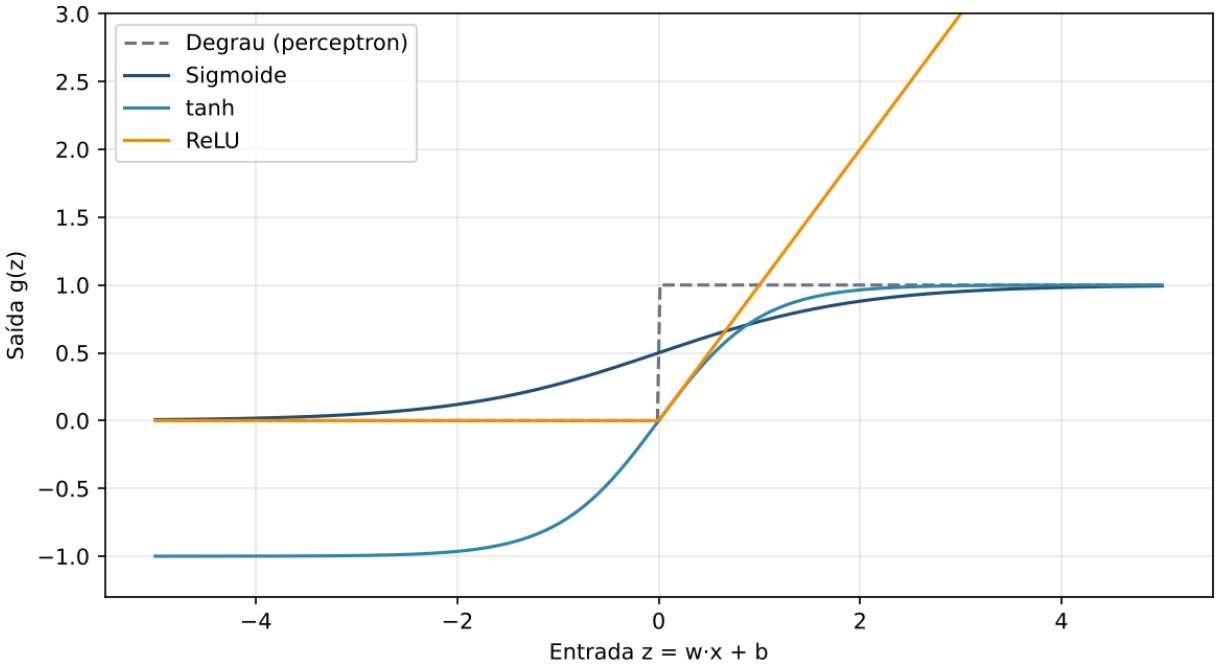
O neurônio artificial

> **Para começar**
> Um comitê decide se haverá churrasco: cada membro opina (vai chover? tem dinheiro?), algumas opiniões pesam mais, e se a soma passa de um limite a resposta é "sim". Um neurônio artificial é isso: soma entradas multiplicadas por pesos e decide quanto "disparar".



Um neurônio artificial: soma ponderada das entradas mais viés, seguida de função de ativação. Fonte: elaboração própria a partir de "The Anatomy of a Neural Network" (Notable Capital) e "Inside a Neuron: The Building Blocks of a Neural Network & AI" (IBM Technology).

Formalmente, $z = \mathbf{w}\cdot\mathbf{x} + b$ e $\hat{y} = g(z)$. O vídeo Inside a Neuron: The Building Blocks of a Neural Network & AI usa preço de imóvel: entradas como área e quartos, pesos dizendo o quanto cada uma importa, viés como deslocamento. "Treinar" é ajustar pesos e viés a partir de dados.



Funções de ativação: o degrau não é diferenciável; sigmoide e tanh saturam; a ReLU não satura para $z > 0$. Fonte: elaboração própria.

Função	Fórmula	Uso / armadilha
Degrau	1 se $z \geq 0$, senão 0	Perceptron original; gradiente zero impede o treino
Sigmoide	$1/(1+e^{-z})$	Saída binária; satura
ReLU	$\max(0, z)$	Padrão em camadas ocultas
GELU / SiLU	variantes suaves	Comuns em Transformers
Softmax	$e^{z_i}/\Sigma_j e^{z_j}$	Saída multiclasse (probabilidades)

A ativação precisa ser **não linear**: compor transformações lineares dá outra transformação linear, e uma rede de cem camadas lineares equivale a uma só.

O perceptron de Rosenblatt (1958) aprendia fronteiras lineares; em 1969, Minsky e Papert mostraram que ele não resolve o XOR, o que contribuiu para o primeiro "inverno da IA". A solução é empilhar camadas: uma **rede multicamada** (MLP) com uma camada oculta já resolve o XOR, e o **teorema da aproximação universal** garante que uma camada larga o bastante aproxima qualquer função contínua. No exemplo do cão, as primeiras camadas detectam cor e bordas, depois contornos, patas e cauda, até o nó de saída "cão".

Você sabia?

Rosenblatt construiu o perceptron em hardware, o Mark I Perceptron, com 400 fotocélulas como "retina". O New York Times noticiou a máquina como o embrião de um computador que um dia andaria, falaria e teria consciência de si.

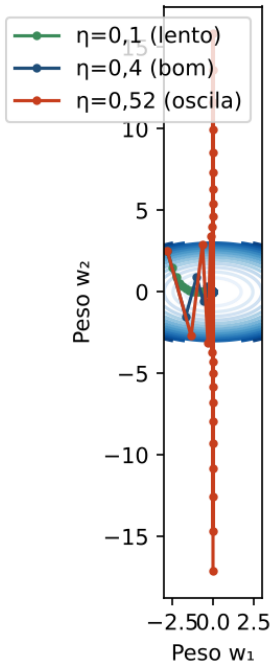
Como uma rede aprende: perda, gradiente e retropropagação

> **Para começar**
> Imagine descer uma montanha de olhos vendados, sentindo com os pés para que lado o chão desce. Treinar uma rede é isso: o chão é o erro (a **perda**), a inclinação é o **gradiente** e cada passo é um pequeno ajuste nos pesos.

Escolhida a perda L(w) (erro quadrático médio para regressão, entropia cruzada para classificação), anda-se no sentido oposto ao gradiente:

$$\mathbf{w \leftarrow w - \eta \cdot \nabla L(w)}$$

η é a **taxa de aprendizado**: pequena demais, o treino é lento; grande demais, oscila ou diverge, como mostra a figura a seguir. Na prática, calcula-se o gradiente em **mini-batches** (dezenas a milhares de exemplos) e usa-se o otimizador Adam/AdamW, padrão em LLMs.



Gradiente descendente em L = ½w₁² + 2w₂² com três taxas de aprendizado: lenta, adequada e instável. Fonte: simulação própria.

Calcular o gradiente de milhões de pesos é viável graças à **retropropagação**, popularizada por Rumelhart, Hinton e Williams em 1986: a regra da cadeia aplicada de trás para frente. Para um peso da última camada, $\partial L/\partial w = (\hat{y} - y) \cdot g'(z) \cdot x$; cada camada reaproveita o gradiente da seguinte, e o custo do caminho de volta é da mesma ordem do de ida. PyTorch e TensorFlow fazem isso por **diferenciação automática**. O loop de treino descrito em What is PyTorch? (Machine/Deep Learning) é sempre o mesmo:

```
for x, y in dataloader:          # batches embaralhados
    y_hat = model(x)             # forward
    loss = loss_fn(y_hat, y)     # perda
    optimizer.zero_grad()
    loss.backward()              # autograd = backprop
    optimizer.step()             # gradiente descendente
```

Problemas clássicos: o **gradiente que desaparece ou explode** ao multiplicar muitas derivadas (remédios: ReLU, boa inicialização, normalização, conexões residuais) e o overfitting (dropout, weight decay, early stopping). Redes não são incompreensíveis em princípio, pois cada número pode ser inspecionado; o que as torna opacas é a escala, e ver cada número não é o mesmo que entender o algoritmo que a rede implementa. Por isso, onde a interpretabilidade é requisito (saúde, crédito), o ML clássico costuma ser preferido.

Machine learning clássico vs. deep learning

> **Para começar**
>
> Para prever inadimplência a partir de uma planilha, uma árvore de decisão costuma ser rápida, barata e explicável. Para reconhecer rostos ou entender frases, redes profundas ganham com folga, mas cobram caro em dados, máquinas e energia.

Critério	ML clássico	Deep learning
Features	Engenharia manual	Aprendidas automaticamente
Dados	Funciona com menos; satura	“Faminto”; continua melhorando
Hardware	CPU costuma bastar	GPU/TPU
Interpretabilidade	Alta	Baixa
Onde brilha	Dados tabulares, finanças	Imagem, áudio, linguagem

Com modelos pré-treinados e *transfer learning*, “DL precisa de muitos dados” vale para treinar do zero; para ajustar um modelo pronto, centenas de exemplos podem bastar. O custo, porém, é real: o vídeo Scaling AI Model Training and Inferencing Efficiently with PyTorch cita um Llama de 7 bilhões de parâmetros treinado com 2 trilhões de tokens em cerca de 200 mil horas de GPU, com técnicas como DDP (uma réplica por GPU) e FSDP (parâmetros fatiados entre GPUs). E a **inferência** domina o custo ao longo da vida do modelo, cerca de 90% segundo AI Inference: The Secret to AI's Superpowers; por isso se usam poda e **quantização** (ex.: FP32 → INT8).

> **Dica**
>
> Conta rápida de memória: parâmetros × bytes por parâmetro. 7B em FP16 = 14 GB; em INT4 ≈ 3,5 GB, mais o espaço do KV cache e das ativações.

Redes convolucionais (CNNs)

> **Para começar**
> Ao procurar um rosto numa foto, seus olhos buscam padrões locais: um olho, uma boca, uma borda. Uma rede convolucional faz o mesmo com uma "lupa" pequena (o filtro) que desliza pela imagem procurando sempre o mesmo padrão.

Uma imagem 64×64 RGB tem 12.288 valores; uma camada densa de 1.000 neurônios teria 12 milhões de pesos e ignoraria que pixels vizinhos se relacionam. As **CNNs** usam **conectividade local** e **compartilhamento de pesos** (o mesmo filtro em toda a imagem). Existem desde os anos 1980 (LeNet, de LeCun), mas explodiram em 2012 com a **AlexNet**, treinada em GPU, que venceu o ImageNet por larga margem. O CNN Explainer (Jay Wang et al.) mostra tudo isso de forma interativa no navegador, com a arquitetura **Tiny VGG**.

Cada neurônio convolucional faz o produto elemento a elemento do kernel com uma janela da entrada, soma, desliza e repete. Os hiperparâmetros são o tamanho do kernel (3×3 é comum: dois 3×3 empilhados veem o mesmo que um 5×5 com menos parâmetros), o **stride** (passo) e o **padding** (bordas extras). A saída tem dimensão **O = ⌊(N − K + 2P) / S⌋ + 1**: entrada 64, kernel 3, sem padding, stride 1 → 62.

```
import numpy as np
def conv2d(img, k, stride=1):
    H, W = img.shape; kh, kw = k.shape
    out = np.zeros(((H-kh)//stride+1, (W-kw)//stride+1))
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            r, c = i*stride, j*stride
            out[i, j] = (img[r:r+kh, c:c+kw] * k).sum()
    return out
sobel_x = np.array([[-1,0,1],[-2,0,2],[-1,0,1]])  # detector de borda vertical
```

Depois de cada convolução vem a **ReLU**; o **max-pooling** 2×2 mantém o maior valor de cada bloco, reduzindo computação e overfitting; o **flatten** transforma o tensor em vetor; e o **softmax** converte os escores finais (**logits**) em probabilidades, dando peso desproporcional ao maior, um "argmax suave".

Camada (Tiny VGG)	Forma da saída
input	64×64×3
conv_1_1 + relu → conv_1_2 + relu	62×62×10 → 60×60×10
max_pool_1	30×30×10
conv_2_1 + relu → conv_2_2 + relu	28×28×10 → 26×26×10
max_pool_2	13×13×10
flatten → dense → softmax	1.690 → 10 classes

> **Dica**
> `CrossEntropyLoss` do PyTorch já aplica o softmax e espera logits crus; aplicar softmax antes é um bug comum.

CNNs são usadas em classificação, segmentação e detecção de objetos. Uma variante em forma de U, a U-Net, é a peça que remove ruído nos modelos de difusão, descritos mais adiante.

> **Você sabia?**
> Hinton, LeCun e Bengio dividiram o Prêmio Turing de 2018 pelo trabalho que tornou as redes profundas práticas. Nos anos 1990, redes convolucionais de LeCun já liam cheques em bancos dos EUA.

Redes recorrentes e sequências

> **Para começar**
> Para entender o fim de uma frase é preciso lembrar o começo. Uma rede recorrente carrega uma "memória de curto prazo" de um passo para o seguinte, como quem lê lembrando do que veio antes.

Uma **RNN** mantém um **estado oculto** $h_t = \tanh(W_h\, h_{t-1} + W_x\, x_t + b)$. O vídeo The Power of Recurrent Neural Networks (RNN) descreve usos como legenda de imagem (um-para-muitos), análise de sentimento (muitos-para-um) e tradução (muitos-para-muitos). Ao longo de muitos passos o gradiente desaparece; a **LSTM** (1997) resolve parcialmente com uma célula de memória e portões. Em 2017, o Transformer trocou a recorrência por **atenção**, que olha todas as posições em paralelo e calcula a relevância entre palavras, tema do Capítulo 2.

Uma linha do tempo das arquiteturas

Ano	Marco	Ideia central
1958	Perceptron (Rosenblatt)	Neurônio com pesos aprendidos
1986	Retropropagação	Treinar redes multicamada
1989–98	CNN / LeNet	Convolução e pesos compartilhados
1991	Mixture of Experts	Especialistas + roteador
1997	LSTM	Memória de longo prazo em RNNs
2012	AlexNet	CNN profunda treinada em GPU
2014	GAN	Gerador vs. discriminador
2015/2020	Difusão (Sohl-Dickstein; DDPM)	Gerar removendo ruído passo a passo
2017	Transformer	Atenção sem recorrência
2018	GPT e BERT	Pré-treino em larga escala
2021	CLIP	Texto e imagem no mesmo espaço

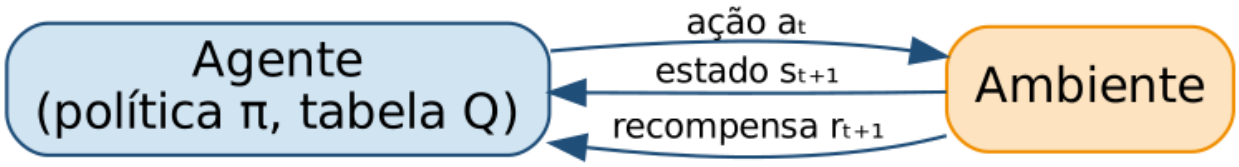
- **Difusão** (Diffusion Models for AI Image Generation): soma-se ruído gaussiano à imagem por cerca de 1.000 passos até restar estática; uma U-Net aprende a prever e remover esse ruído, guiada pelo texto do prompt.
- **Redes neurais de grafos** (Graph Neural Networks Explained: A Clear Guide to GNN Basics & Models): generalizam a convolução para grafos com troca de mensagens entre nós vizinhos.

- **Mixture of Experts** (What is Mixture of Experts?): um roteador ativa só alguns especialistas por token (2 de 8 no Mixtral 8x7B), escalando parâmetros sem escalar o cômputo na mesma proporção.
- **Multimodalidade** (What is Multimodal AI? How LLMs Process Text, Images, and More): um codificador de visão acoplado a um LLM ou um modelo nativo que trata imagens como sequências de patches.

Aprendizado por reforço

> **Para começar**
> Para treinar um cachorro, você não explica a regra: dá um petisco quando ele acerta. No aprendizado por reforço, um **agente** tenta ações, recebe pontos ou penalidades e ajusta o comportamento para ganhar mais ao longo do tempo.

As palestras do CS50 (como CS50x em Português - Inteligência Artificial) mostram uma IA de Breakout que "descobre" sozinha que furar a fileira de cima e deixar a bola quicar atrás dos tijolos maximiza a pontuação. Formalmente, o problema é um **processo de decisão de Markov**: estados, ações, transições e recompensas. A **política** diz que ação tomar em cada estado, e o objetivo é maximizar o retorno descontado $G = r_1 + \gamma r_2 + \gamma^2 r_3 + \ldots$, com $0 \le \gamma < 1$.



O ciclo agente–ambiente do aprendizado por reforço. Fonte: elaboração própria a partir do CS50 e de Rouhard (2020).

O **Q-learning** aprende Q(s, a), o valor de tomar a ação a no estado s e agir bem depois:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \right]$$

A atualização compara a nova estimativa (recompensa mais o melhor valor futuro, a **equação de Bellman**) com a antiga. No CS50 AI, o agente joga Nim milhares de vezes contra si mesmo e termina imbatível.

```
import random
def escolher(Q, s, acoes, eps=0.1):
    if random.random() < eps:                # explorar
        return random.choice(acoes)
    return max(acoes, key=lambda a: Q.get((s, a), 0))   # aproveitar

def atualizar(Q, s, a, r, s2, acoes2, alpha=0.5, gamma=0.9):
    futuro = max((Q.get((s2, a2), 0) for a2 in acoes2), default=0)
    antigo = Q.get((s, a), 0)
    Q[(s, a)] = antigo + alpha * (r + gamma * futuro - antigo)
```

O termo entre colchetes é chamado de **erro de diferença temporal**, e α é a taxa de aprendizado. A mesma lógica vale para problemas muito maiores, desde que o agente possa experimentar muitas vezes.

Se o agente sempre escolher o melhor conhecido, nunca descobre alternativas, como quem sempre pede o mesmo prato no restaurante. A política **ε-greedy** escolhe uma ação aleatória com probabilidade ε e, no resto do tempo, a melhor conhecida. Quando há estados demais para uma tabela, uma rede neural estima Q: é o **DQN** da DeepMind, que aprendeu dezenas de jogos de Atari a partir dos pixels. Variantes como o PPO são usadas no RLHF, que alinha LLMs com feedback humano.

> **Você sabia?**
> Em 2016, o AlphaGo venceu Lee Sedol por 4 a 1. Na segunda partida, o lance 37 parecia um erro para os comentaristas e acabou decisivo: a máquina estimara que um humano jogaria ali com chance de cerca de 1 em 10 mil.

Deep learning no navegador: TensorFlow.js

> **Para começar**
> Para cozinhar não é preciso fabricar o fogão. Bibliotecas como TensorFlow e PyTorch cuidam da matemática pesada e do uso da GPU, e algumas rodam até no navegador, sem enviar seus dados a servidor nenhum.

O **TensorFlow** (Google, versão 1.0 em 2017) opera sobre **tensores**, arrays n-dimensionais, e tem a API de alto nível Keras (What is TensorFlow?). O **TensorFlow.js**, segundo o artigo “Introduction to TensorFlow.js”, de Pierre Rouhard (2020), traz três vantagens no navegador: **privacidade** (os dados não saem do dispositivo), baixa latência e interatividade com câmera e microfone. A aceleração vem do WebGL e, mais recentemente, do WebGPU.

O estudo de caso de Rouhard é o **Mountain Car**: um carro num vale precisa chegar ao topo, mas o motor é fraco demais para subir direto; a solução é ir e voltar para ganhar impulso, algo que um agente guloso nunca descobriria sem exploração. O modelo é uma rede rasa que recebe posição e velocidade e devolve um Q para cada uma das três ações:

```
const model = tf.sequential();
model.add(tf.layers.dense({inputShape: [2], units: 50, activation: 'relu'}));
model.add(tf.layers.dense({units: 3}));               // Q(s, a) para 3 ações
model.compile({loss: 'meanSquaredError', optimizer: tf.train.adam(1e-3)});
```

A cada episódio, o agente escolhe ações com ε-greedy, guarda cada transição (estado, ação, recompensa, próximo estado) numa **memória de replay** e, de tempos em tempos, treina a rede com amostras aleatórias dessa memória, usando a equação de Bellman para calcular os alvos. Rouhard também ajusta a recompensa para crescer com a posição do carro, em vez de premiar só o topo, uma técnica chamada *reward shaping*.

> **Atenção**
> Tensores no backend WebGL vivem na GPU e não são coletados pelo garbage collector do JavaScript. Envolva cálculos em `tf.tidy()` ou chame `dispose()`, senão a memória vaza a cada passo.

Fechando o ciclo

As palestras do CS50 terminam onde o próximo capítulo começa: **embeddings** (palavras como vetores, com “breakfast”, “lunch” e “dinner” próximas no espaço), **atenção** e Transformers que geram um token por vez. Os vídeos 7 AI Terms You Need to Know: Agents, RAG, ASI & More e What is Superalignment? completam o vocabulário: RAG, IA agêntica, MCP e, no horizonte, AGI, ASI e o desafio de manter sistemas mais capazes que humanos alinhados a valores humanos.

> **Em resumo**
>
> Busca encontra caminhos; lógica deduz; probabilidade lida com incerteza; otimização encontra configurações; ML aprende com exemplos; redes neurais aprendem por gradiente e retropropagação; CNNs exploram a estrutura espacial; RL aprende a agir por tentativa e erro. Os LLMs usam quase todos esses ingredientes.

Fontes deste capítulo

Título	URL	Canal / autor	Ponto principal
Harvard CS50’s Artificial Intelligence with Python – Full University Course	https://www.youtube.com/watch?v=5NgNicANyqM	freeCodeCamp.org	Busca, lógica, incerteza, otimização, ML e redes
CS50 for Business - Lecture 4 - Approaching Artificial Intelligence	https://www.youtube.com/watch?v=pFq5szYIWQk	CS50	Minimax, explosão combinatória e reforço
CS50x em Português - Inteligência Artificial	https://www.youtube.com/watch?v=YFzkVvjiPmM	CS50	Aula introdutória de IA em português
Scaling AI Model Training and Inferencing Efficiently with PyTorch	https://www.youtube.com/watch?v=85RfazjDPwA	IBM Technology	DDP, FSDP e torch.compile
Graph Neural Networks Explained: A Clear Guide to GNN Basics & Models	https://www.youtube.com/watch?v=eGoszzMkGfU	IBM Technology	Troca de mensagens em grafos
Confusion to Clarity: Mastering Confusion Matrix in Machine Learning	https://www.youtube.com/watch?v=PoqGrCscJ7k	IBM Technology	Matriz de confusão, precisão e recall
What is PyTorch? (Machine/Deep Learning)	https://www.youtube.com/watch?v=fJ40w_2h8kk	IBM Technology	Loop de treino com autograd
7 AI Terms You Need to Know: Agents, RAG, ASI & More	https://www.youtube.com/watch?v=VSFuqMh4hus	IBM Technology	Vocabulário: agentes, RAG, MCP, ASI
Diffusion Models for AI Image Generation	https://www.youtube.com/watch?v=x2GRE-RzmD8	IBM Technology	Difusão direta e reversa com U-Net
Predictive vs Generative AI: How They Work and When to Use Each	https://www.youtube.com/watch?v=phOhGqpXss4	IBM Technology	IA preditiva vs. generativa
AI Inference: The Secret to AI's Superpowers	https://www.youtube.com/watch?v=XtT5i0ZeHHE	IBM Technology	Inferência domina o custo; quantização
What is Semi-Supervised Learning?	https://www.youtube.com/watch?v=C3Lr6Waw66g	IBM Technology	Pseudo-rótulos, clustering, active learning
What is Zero-Shot Learning?	https://www.youtube.com/watch?v=pVpr4GYLzAo	IBM Technology	Classificar classes nunca vistas
How Linear Algebra Powers Machine Learning (ML)	https://www.youtube.com/watch?v=-KKxtHwxKhE	IBM Technology	Vetores, cosseno e SVD
Algorithmic Bias in AI: What It Is and How to Fix It	https://www.youtube.com/watch?v=og67qeTZPYs	IBM Technology	Viés vem sobretudo dos dados
What is Multimodal AI? How LLMs Process Text, Images, and More	https://www.youtube.com/watch?v=J51oZYcNvP8	IBM Technology	Multimodalidade modular vs. nativa
What is Mixture of Experts?	https://www.youtube.com/watch?v=sYDlVVyJYn4	IBM Technology	Roteamento top-k e esparsidade
What is Superalignment?	https://www.youtube.com/watch?v=N_RLQ56d3Z4	IBM Technology	AGI, ASI e técnicas de alinhamento

Título	URL	Canal / autor	Ponto principal
Inside a Neuron: The Building Blocks of a Neural Network & AI	https://www.youtube.com/watch?v=H_jD5xa1nRg	IBM Technology	Soma ponderada, viés e ativação
The Power of Recurrent Neural Networks (RNN)	https://www.youtube.com/watch?v=Gafjk7_w1i8	IBM Technology	Estado oculto, LSTM e GRU
What is TensorFlow?	https://www.youtube.com/watch?v=GnGhI1vKi20	IBM Technology	Tensores, Keras e deploy
Text Classification: AI Techniques and Real-World Applications	https://www.youtube.com/watch?v=hHiPs_wICsE	IBM Technology	Tipos de classificação, pipeline e drift
Deep Learning vs Machine Learning	—	Matt Crabtree (DataCamp, 2024)	IA ⊃ ML ⊃ DL e comparação
The Anatomy of a Neural Network	—	Rares Crisan (Notable Capital, 2023)	Neurônio, pesos, backprop e linha do tempo
CNN Explainer	https://poloclub.github.io/cnn-explainer/	Jay Wang et al. (Georgia Tech / Oregon State)	Tiny VGG: convolução, pooling e softmax
Introduction to TensorFlow.js	—	Pierre Rouhard (Medium, 2020)	Mountain Car com Q-learning no navegador

CAPÍTULO 2 ·

LLMs, Transformers e Mistura de Especialistas

> *Um modelo de linguagem grande é uma função que recebe uma sequência de números e devolve uma distribuição de probabilidade sobre o próximo número. A descrição é correta e quase inútil para entender o que acontece lá dentro. Este capítulo percorre o caminho completo: do texto ao token, do token ao vetor, do vetor à atenção, da atenção ao modelo treinado e do modelo ao produto em uso em out/2026. O fio condutor são os experimentos publicados por Fabio Akita no blog AkitaOnRails: a linha do tempo de GPT-2 a 2026, o treino de um modelo de 337 milhões de parâmetros e o benchmark v4 de modelos abertos.*

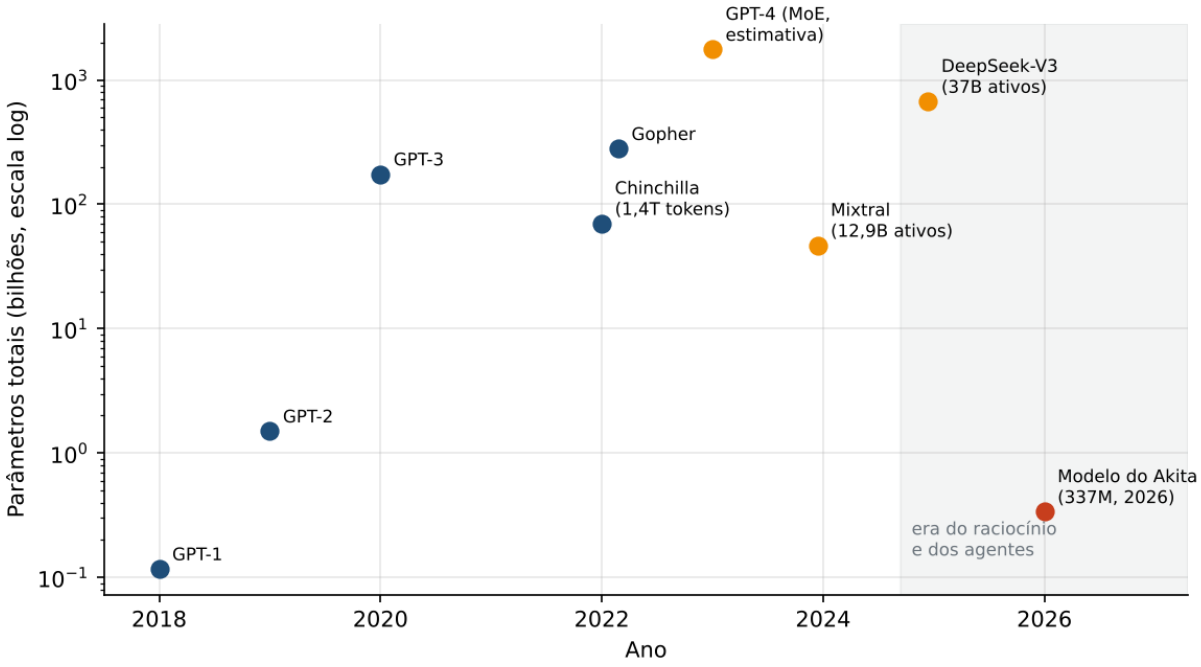
Para começar

Pense na sugestão de próxima palavra do teclado do celular. Um LLM (*Large Language Model*) faz o mesmo, treinado com uma parte enorme da internet e levando em conta tudo o que veio antes. Repetindo "qual é a próxima palavra?" milhares de vezes, ele escreve respostas, código e resumos.

Uma linha do tempo curta: de GPT-2 à era agêntica

A história dos LLMs tem viradas claras: primeiro os modelos ficaram maiores, depois aprenderam a "pensar" antes de responder e, por último, ganharam ferramentas em volta. O post LLMs + Agentes = competente o suficiente (Akita, 12/10/2026) organiza essas fases:

Ano	Marco	O que mudou
2017	"Attention Is All You Need"	O Transformer processa a sequência em paralelo; cada token "olha" para todos os outros.
2018–19	GPT-1, BERT, GPT-2 (1,5B)	Pré-treino em texto não rotulado + ajuste fino; texto longo coerente.
2020	GPT-3 (175B)	Aprendizado *few-shot* no prompt; leis de escala.
2022	Chinchilla; ChatGPT	Modelos estavam subtreinados em dados; RLHF chega ao público.
2023	GPT-4	Mistura de especialistas (segundo relatos) e multimodalidade.
2024	o1 (OpenAI)	RL para raciocínio e *test-time compute*.
2025	DeepSeek-V3/R1	Eficiência: 671B totais com 37B ativos; raciocínio emergindo de RL.
2026	MCP e harnesses agênticos	O ganho vem do sistema em volta do modelo.



Parâmetros totais de modelos marcantes (escala log). Em laranja, modelos MoE, cujo custo por token depende dos parâmetros *ativos*. Fonte: elaboração própria com dados do blog AkitaOnRails (12/10 e 14/10/2026) e de *Transforming Language with Generative Pre-trained Transformers (GPT)*.

A tese do Akita é que os modelos de base **convergiram**: o Stanford AI Index mostra o gap entre o 1º e o 10º colocado no Chatbot Arena caindo de 12% para 5% em um ano. O podcast New frontier AI models, TypeSafe's Jev AI, & NASA's IBM collab chega ao mesmo lugar: a disputa passou de "inteligência do modelo" para "inteligência do sistema" (memória, ferramentas, execução de código). 2023–24 foram os anos dos modelos maiores, 2025 o do raciocínio, 2026 o do **raciocínio por unidade de compute**.

Do texto aos números: tokens

> **Para começar**
> Computador não lê letras, só faz contas. Por isso o texto é cortado em pedaços chamados **tokens** e cada pedaço recebe um número. Tudo é medido em tokens: o preço da API, o tamanho máximo do texto e a velocidade da resposta.

Há três granularidades possíveis, como explica o vídeo O que são tokens?, de Fernanda Kipper. **Caracteres** geram sequências enormes, e a atenção custa $O(n^2)$ no comprimento. **Palavras inteiras** explodem o vocabulário com flexões ("corre", "corria", "corríamos"). **Subpalavras** são o meio-termo usado por todos os LLMs: radicais e sufixos frequentes viram tokens; palavras raras se decompõem em pedaços.

O algoritmo mais comum é o **BPE** (*Byte-Pair Encoding*): começa com bytes e funde repetidamente o par adjacente mais frequente do corpus até atingir o tamanho de vocabulário desejado. Partir de bytes garante que qualquer string, até emoji, seja representável.

“Um token” não é uma unidade universal. Na demonstração do mesmo vídeo, uma frase em português deu 22 tokens no GPT-4o e 15 na versão em inglês. Duas lições: vocabulário maior gera menos tokens por texto, e **inglês sai mais barato**, porque domina os dados de treino; em português paga-se um “imposto de idioma” de 30–50%. Conte sempre no tokenizer do modelo que vai usar.

Os tokenizers mais conhecidos seguem a mesma ideia com critérios diferentes: WordPiece no BERT, SentencePiece no T5 e nas primeiras versões do Llama, BPE em nível de bytes na família GPT. O tamanho do vocabulário é uma escolha de projeto: o tokenizer de 32.768 tokens do modelo de estudo do Akita é pequeno; o o200k do GPT-4o tem cerca de 200 mil.

Por que a saída custa mais caro

APIs cobram 3–5× mais pela saída (Opus: US$ 5 de entrada e US$ 25 de saída por milhão de tokens). A razão está nas duas fases da inferência:

- **Prefill** (entrada): todos os tokens do prompt são processados de uma vez, em paralelo; limitado por *compute*.
- **Decode** (saída): um token por passada, e cada passada lê **todos os pesos** da memória; limitado por *largura de banda de memória*.

O **KV cache** guarda as chaves e valores de atenção já calculados, então o contexto não é recalculado; o custo vem de a geração ser serial e presa à banda. O vídeo ¿Es El Fin De Los LLM? (NCM) faz a conta para um modelo de 16 GB numa GPU de 1.792 GB/s: gerar 1.024 tokens move ~16 TB de memória, cerca de 9 segundos.

> **Dica**
> Velocidade máxima de decode para um usuário: tok/s ≈ largura de banda (GB/s) ÷ bytes lidos por token (pesos ativos + KV cache). Para rodar modelos localmente, banda de memória vale mais que TFLOPS.

Dos números ao significado: embeddings

> **Para começar**
> Um número de token é como o número de uma casa: não diz nada sobre quem mora lá. Para o modelo entender significado, cada token vira um **embedding**, um endereço num mapa em que palavras de sentido parecido moram no mesmo bairro.

A primeira camada do modelo é a **matriz de embeddings**, com uma linha por token, que transforma cada ID num vetor denso (768 dimensões no GPT-2; de 4 mil a mais de 12 mil nos modelos de fronteira). Os vetores começam aleatórios e o treino os arranja de modo que tokens usados em contextos parecidos fiquem próximos (veja Se você só pode assistir UM vídeo sobre LLMs, assista esse.). A evolução, contada em What are Word Embeddings?:

Família	Exemplo	Limitação
Frequência	Bag-of-words, TF-IDF	Sem semântica: “carro” e “automóvel” não se relacionam
Preditivos estáticos	Word2Vec (2013), GloVe (2014)	Um vetor por palavra: “banco” de dinheiro e de praça coincidem
Contextuais	ELMo, BERT, GPT	O vetor depende da frase; custo maior

A similaridade costuma ser medida por **cosseno**, o mesmo cálculo da busca vetorial em RAG, tema do Livro 2, Capítulo 3 (RAG).

Você sabia?

O Word2Vec ficou famoso pelas analogias feitas com aritmética de vetores: “rei” − “homem” + “mulher” cai perto de “rainha”. Ninguém ensinou gênero ao modelo; a relação emergiu só de prever palavras pelo contexto.

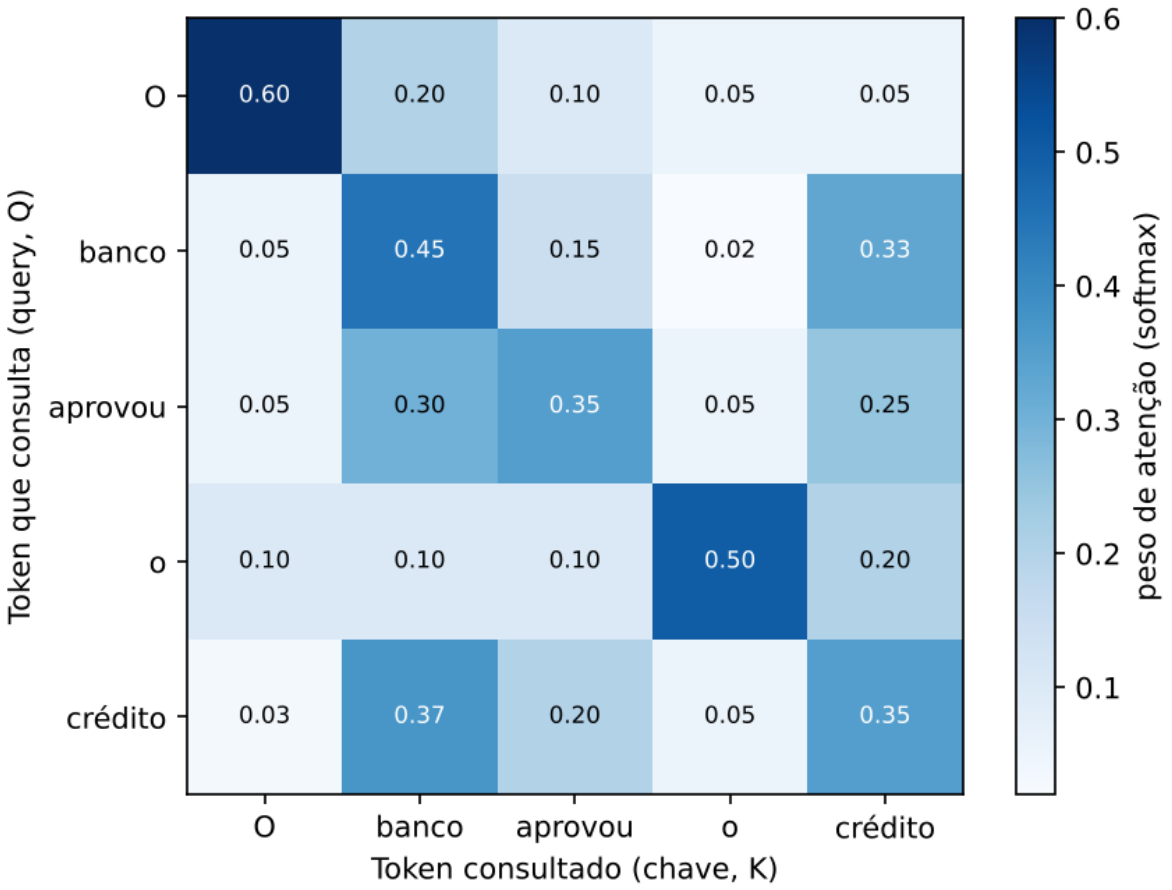
A posição importa

“A mãe matou a filha” e “a filha matou a mãe” têm os mesmos tokens. Como a atenção é indiferente à ordem, a posição precisa ser injetada. O paper original somava senóides fixas; hoje o padrão é **RoPE** (*Rotary Position Embedding*), que rotaciona os vetores por um ângulo proporcional à posição, de modo que a atenção dependa da distância relativa. É o RoPE que permite técnicas de extensão de contexto.

Atenção: o mecanismo que destravou tudo

Para começar

Ao ler “o banco aprovou o crédito”, você sabe que é banco de dinheiro porque olhou para “crédito”. **Atenção** é isso: cada palavra procura, nas outras, as pistas que esclarecem seu sentido.



Matriz de atenção ilustrativa (valores didáticos) para “O banco aprovou o crédito”. Cada linha soma 1; “banco” consulta fortemente “crédito”. Num modelo decoder real, um token só enxerga os anteriores. Fonte: elaboração própria a partir do exemplo de *Se você só pode assistir UM vídeo sobre LLMs, assista esse.*

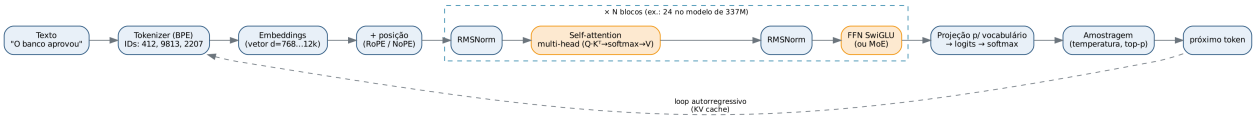
Para cada token, três projeções aprendidas geram **Q** (*query*, “o que procuro”), **K** (*key*, “o rótulo que ofereço”) e **V** (*value*, “o conteúdo que entrego”):

```
Attention(Q, K, V) = softmax( Q·Kᵀ / √d_k  +  M ) · V
# M = máscara causal: o token i não vê i+1, i+2, ...
# √d_k evita que produtos escalares grandes saturem o softmax
```

A saída de cada token é a média dos V de todos os tokens, ponderada pela afinidade Q·K; sem V, o mecanismo não entrega informação. Na **multi-head attention**, várias atenções menores rodam em paralelo, cada uma aprendendo um tipo de relação (sintaxe, correferência). Variantes como **GQA** e **MLA** reduzem o tamanho do KV cache, e o **FlashAttention** calcula a mesma conta em blocos sem materializar a matriz n×n.

O bloco Transformer

A arquitetura original era **encoder-decoder**, para tradução (veja AI and Large Language Models Boost Language Translation). Os LLMs generativos atuais são **só decoder**: uma pilha de blocos idênticos, cada um com atenção causal e uma rede feed-forward (FFN), ligados por conexões residuais e normalização.



Pipeline de um LLM decoder-only moderno (RMSNorm, RoPE, SwiGLU). A saída volta à entrada a cada token gerado; o KV cache evita recalcular a atenção dos tokens anteriores. Fonte: elaboração própria com dados do blog AkitaOnRails (14/10/2026).

Um Transformer de 2026 difere do de 2017 em detalhes: normalização antes de cada sub-bloco, **RMSNorm**, ativação **SwiGLU**, RoPE e embeddings compartilhados entre entrada e saída. A FFN concentra cerca de dois terços dos parâmetros, e é nela que entra a mistura de especialistas.

O custo quadrático e as alternativas

A atenção compara todos os pares de tokens: dobrar o contexto quadruplica o compute. Os **state space models** como Mamba mantêm um estado resumido com custo linear, mas perdem em recuperação exata. A IBM, no Granite 4.0 (Granite 4.0: Small AI Models, Big Efficiency), usa um híbrido de 9 blocos Mamba-2 para cada bloco Transformer, com até 80% menos memória.

Gerar texto: o loop autorregressivo e suas alternativas

> **Para começar**
> Gerar texto é como uma brincadeira de completar frases em que cada jogador acrescenta uma palavra olhando tudo o que já foi dito. É a geração **autorregressiva**: funciona muito bem, mas cada palavra espera a anterior.

A saída do último bloco vira probabilidades sobre o vocabulário, amostradas com **temperatura** (achata ou afia a distribuição) e **top-p** (amostra só do menor conjunto de tokens cuja probabilidade soma p). O token escolhido é anexado e o ciclo recomeça. Como lembra Transforming Language with Generative Pre-trained Transformers (GPT), GPT significa *Generative Pre-trained Transformer*, e o ChatGPT é a interface, não o modelo.

Há tentativas de gerar vários tokens por vez. O **NCM** (Stanford, out/2026), analisado em ¿Es El Fin De Los LLM? (NCM) e NCM matou o LLM: dessa vez, parece que é real, comprime o texto numa pirâmide de códigos e gera cada nível numa passada, do esboço ao detalhe. O "31× mais rápido" do anúncio compara com um modelo de **difusão**, não com um autorregressivo de produção. O NCM ganha em Sudoku difícil (86% contra 1% do autorregressivo), perde em tradução e ainda está em escala de brinquedo (725M no maior gerador). Difusão de texto é a alternativa usável hoje, com alguma perda de qualidade.

Outra linha são modelos que nem geram texto. O **Jev**, da TypeSafe (What Is Jev? The AI Model That Doesn't Generate Text), recebe um estado e perguntas tipadas (sim/não, escolha, escala) e devolve todas as probabilidades de uma vez. O Akita (Entendendo o Hype do Jev: pra que serve?) mostra que o "193× mais rápido" vem justamente de trocar um LLM de chat por um classificador.

Como um LLM é treinado

> **Para começar**
> Treinar um LLM lembra formar um profissional: primeiro ele lê uma biblioteca inteira (pré-treino), depois faz um estágio com perguntas e respostas (ajuste fino) e, por fim, recebe avaliações e ajusta o comportamento (aprendizado por reforço).

Pré-treino: prever o próximo token

O objetivo é minimizar a **entropia cruzada** (quão surpreso o modelo fica com o token certo) em trilhões de tokens de texto. É **autossupervisionado**, não "não supervisionado": há um rótulo exato, o próximo token, mas ninguém precisa anotá-lo. O GPT-3 partiu de 45 TB de texto bruto, filtrados para ~570 GB (How Large Language Models Work).

O post Treinei meu próprio modelo de IA (spoiler: é só pra estudo) torna isso concreto: um decoder de 337M de parâmetros (24 camadas, tokenizer BPE de 32.768 tokens) treinado com 10 bilhões de tokens, 60% deles de Python, em cerca de 137 horas numa única GPU de consumidor. Com 2B e 5B tokens vistos, o modelo praticamente não acertava tarefas de código; com 10B, saltou para 65 de 100, a **emergência** típica. Mesmo assim, tirou **zero em tarefas agênticas**: abaixo de certa escala, o modelo não sustenta o loop de ferramentas. Outras lições do post: menor perda não significa melhor código (avalie com tarefas executáveis) e o formato de treino precisa ser o formato de uso.

Leis de escala

Kaplan et al. (2020) mostraram que a perda cai como lei de potência em parâmetros N, dados D e compute C ≈ 6·N·D FLOPs. O **Chinchilla** (Hoffmann et al., 2022) corrigiu a receita: para um orçamento fixo, N e D devem crescer juntos, cerca de 20 tokens por parâmetro; com isso, um modelo de 70B superou o Gopher de 280B. Três ressalvas:

1. **Ótimo de treino não é ótimo de produto.** A inferência é paga milhões de vezes, então compensa sobretreinar modelos pequenos (o Llama 3 8B viu 15T tokens, quase 100× a razão de Chinchilla).
2. **Perda não é capacidade.** Capacidades parecem "emergir" quando a métrica é acertou/errou.
3. **Há uma terceira lei: test-time compute.** Desde o o1, pensar mais tokens melhora respostas, e a inferência passou a dominar o compute de IA, segundo 5 AI Myths & The Truth Behind Them: ML, Context, Agents & More.

A convergência é visível no MMLU (Small vs. Large AI Models: Trade-offs & Use Cases Explained): o menor modelo acima de 60% caiu de 65B (fev/2023) para menos de 3B ativos (mar/2024).

SFT e RLHF

Um modelo base continua documentos; não "responde". O **ajuste fino supervisionado** (SFT) treina com pares instrução→resposta no formato de conversa. Os pesquisadores da

Anthropic, em Interpretability: Understanding how AI models think, descrevem o resultado como um autor que escreve o roteiro de um personagem chamado Assistant.

No **RLHF** (aprendizado por reforço com feedback humano), humanos comparam pares de respostas, um modelo de recompensa aprende essas preferências e o LLM é otimizado para maximizá-las. Como recompensa o que *parece* bom, tem efeitos colaterais: **bajulação** (perguntar "tem certeza?" faz o modelo trocar uma resposta certa; veja GPT-5: Five AI Model Improvements to Address LLM Weaknesses), **excesso de confiança** e **AI slop**, o estilo polido e repetitivo descrito em What is AI Slop? Low-Quality AI Content Causes, Signs, & Fixes.

> **Você sabia?**
> A palavra inglesa "delve" (aprofundar-se) apareceu cerca de 25 vezes mais em artigos de 2024 do que dois anos antes. É uma das marcas estatísticas do texto gerado por modelos ajustados com preferências humanas.

Variantes como o DPO dispensam o modelo de recompensa explícito e otimizam direto sobre pares de preferência. Em todas, uma penalidade impede que o modelo se afaste demais da versão SFT, para não "trapacear" a recompensa com respostas estranhas.

RL de raciocínio

A grande mudança de 2024–25 foi o **RL com recompensas verificáveis** (RLVR): um verificador programático (o teste passou? a conta bate?) dá a recompensa, e o modelo descobre sozinho que cadeias de raciocínio longas, com verificação e retrocesso, acertam mais. A linhagem DeepSeek (What is DeepSeek? AI Model Basics Explained) ilustra:

Modelo	Data	Inovação
V2	jun/2024	Multi-head latent attention + DeepSeekMoE
V3	dez/2024	671B totais, 37B ativos; treino eficiente em ~2.000 GPUs
R1-Zero	jan/2025	RL puro sobre o base: o raciocínio emerge, com baixa legibilidade
R1	jan/2025	SFT inicial + RL; comparável ao o1 em matemática e código

> **Atenção**
> A cifra de "US$ 6 milhões" do DeepSeek-V3 refere-se só ao treino final, sem pesquisa, experimentos e infraestrutura.

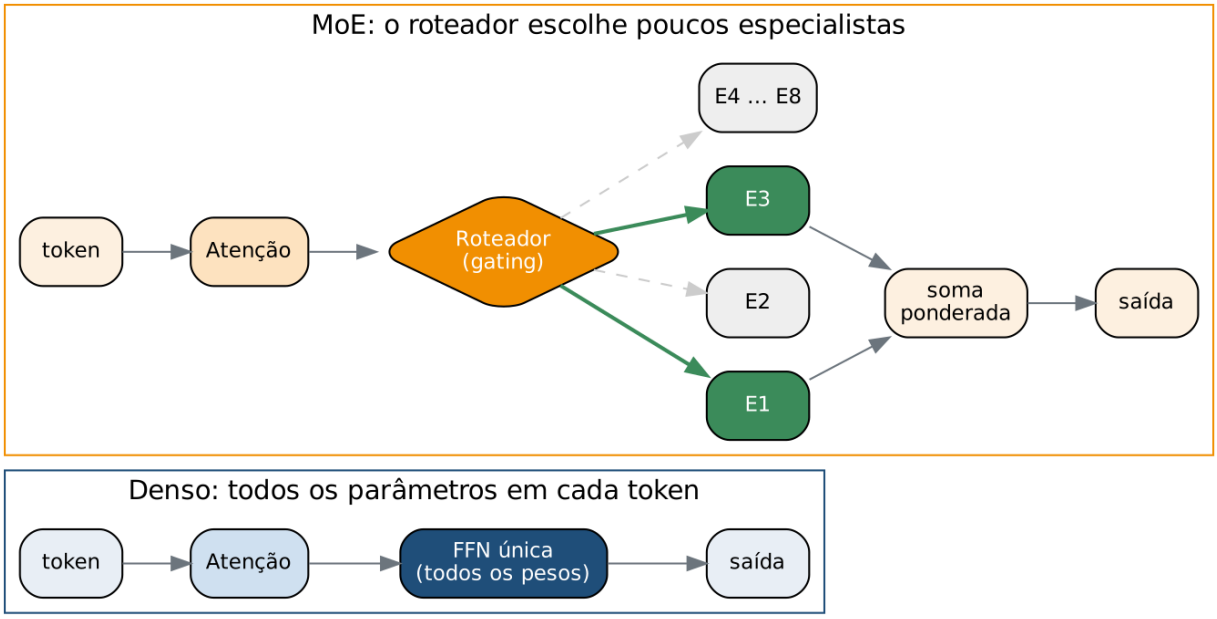
Pensar mais nem sempre é pensar melhor. No vídeo GLM 5.3 é a IA Aberta que Encostou no Claude, o nível padrão de *reasoning effort* foi o mais caro e errático, e o nível máximo foi o pior em detectar premissas falsas; o nível baixo fez o mesmo trabalho com um quarto dos tokens. O mesmo vídeo mostra que o GLM 5.3 manteve a base do 5.2 e mudou só o pós-treino, com o Terminal-Bench saltando de 4,6 para 28,3: o pós-treino virou a alavanca principal.

Mistura de Especialistas e ajuste com LoRA

Para começar

Imagine uma empresa em que todos os 70 mil funcionários precisam ler e assinar cada e-mail, do pedido de café ao contrato milionário. É um modelo **denso**: competente, mas cada e-mail custa o salário de todos. A Mistura de Especialistas põe uma recepção que encaminha cada e-mail só para poucos departamentos.

Num LLM **denso** (GPT-3, Llama 3, Mistral 7B), cada token passa por todos os pesos: um modelo de 70B faz ~140 bilhões de operações por token, seja a pergunta "quanto é 2+2?" ou um teorema. Mais parâmetros trazem mais conhecimento, mas cada token paga por todos, no treino, na latência e no preço.



Denso × MoE. No denso, a FFN inteira trabalha em cada token. No MoE, um roteador ativa só alguns especialistas (em verde); os outros ficam ociosos, mas continuam ocupando memória. Fonte: elaboração própria.

Num **MoE**, a FFN de cada bloco é trocada por N redes menores, os **especialistas**, e um **roteador** dá uma nota a cada um para o token atual; mantêm-se os **top-k** (tipicamente 1 a 8) e a saída é a soma ponderada deles. Muitos modelos acrescentam um **especialista compartilhado** sempre ativo. Os "departamentos" não são legíveis por humanos; são especializações estatísticas aprendidas no treino. A distinção que importa: **parâmetros totais** definem a *memória*; **parâmetros ativos** definem o *compute* por token, portanto velocidade e custo.

Modelo	Totais / ativos	Roteamento
Mixtral 8×7B (dez/2023)	46,7B / 12,9B	8 especialistas, top-2
DeepSeek-V3 / R1	671B / 37B	256 roteados, top-8, + 1 compartilhado
Qwen3-235B-A22B (abr/2025)	235B / 22B	128 especialistas, top-8
gpt-oss-120b (ago/2025)	117B / 5,1B	128 especialistas, top-4; pesos abertos
Llama 4 Maverick (abr/2025)	400B / 17B	128 especialistas + compartilhado

Atenção

Especialistas não são agentes (AI Agents vs Mixture of Experts: AI Workflows Explained). Agentes roteiam tarefas na aplicação; o MoE roteia tokens dentro de uma camada, e a escolha muda a cada token e a cada camada.

O MoE não é almoço grátis. Ganha em qualidade por FLOP, mas perde em qualidade por GB: o modelo **inteiro** precisa caber na memória. Exige perdas auxiliares para que o roteador não concentre o tráfego em poucos especialistas, é menos estável no treino e pede comunicação intensa entre GPUs. Em compensação, permite **offload**: no llama.cpp, `--cpu-moe` mantém os especialistas na RAM e só atenção, roteador e KV cache na GPU, e motores como o Strata (descrito em Melhor LLM Open Source: Strata com Qwen 3.8 Flash Next) tratam a VRAM como cache dos especialistas mais usados. Assim, um MoE de mais de 100B roda numa GPU de consumidor.

Dica

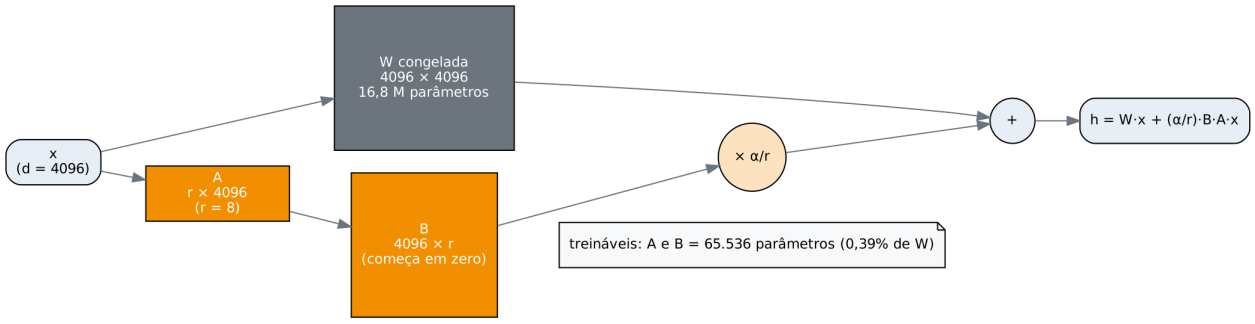
Para escolher um MoE local, faça duas contas: **cabe?** (parâmetros totais × bytes por peso, mais KV cache, contra VRAM + RAM) e **é rápido?** (parâmetros ativos × bytes por peso, contra a banda da memória). Um MoE de 30B com 3B ativos em 4 bits ocupa ~17 GB, mas lê só ~1,7 GB por token.

Você sabia?

A ideia de MoE é de 1991: o artigo *Adaptive Mixtures of Local Experts*, de Jacobs, Jordan, Nowlan e Geoffrey Hinton, já propunha uma rede "porteira" escolhendo especialistas. Ela voltou em escala em 2017, com a camada MoE esparsa de Noam Shazeer, coautor do paper do Transformer.

LoRA: ajustar sem mexer no modelo inteiro

O fine-tuning completo atualiza todos os pesos: com Adam, cada parâmetro custa ~16 bytes, e um 7B pede ~112 GB. O **LoRA** (*Low-Rank Adaptation*, Hu et al., Microsoft, 2021) parte da observação de que a *mudança* necessária nos pesos tem posto baixo. Congela-se W e treina-se ΔW = **B·A**, duas matrizes finas de posto **r** pequeno (4 a 64): h = W·x + (α/r)·B·A·x. B começa em zero, então o modelo começa idêntico ao original.



LoRA numa projeção de 4096×4096: W fica congelada e só A e B são treinadas. Com r = 8, são 65.536 parâmetros treináveis contra 16,8 milhões de W (0,39%). Fonte: elaboração própria a partir de Hu et al. (2021).

No GPT-3 175B, o artigo original relata 10.000× menos parâmetros treináveis e 3× menos memória, sem latência extra (B·A pode ser somada a W). O **QLoRA** (Dettmers et al.,

2023) carrega o base em 4 bits e treina os adapters por cima: um 65B ajustado numa GPU de 48 GB, um 7B numa placa de 8–12 GB. Como o base não muda, um servidor pode carregar vários adapters e trocá-los por requisição. Bibliotecas de referência: PEFT (Hugging Face) e Unsloth.

Necessidade	Melhor ferramenta
Instrução, tom, poucos exemplos	Prompt; veja Livro 2, Capítulo 1 (Prompts na prática)
Fatos que mudam, documentos internos	RAG; veja Livro 2, Capítulo 3 (RAG). LoRA não serve para “decorar” fatos
Formato rígido, estilo, classificação em volume	LoRA/QLoRA sobre um modelo aberto
Comportamento ou idioma totalmente novo	Fine-tuning completo ou pré-treino continuado

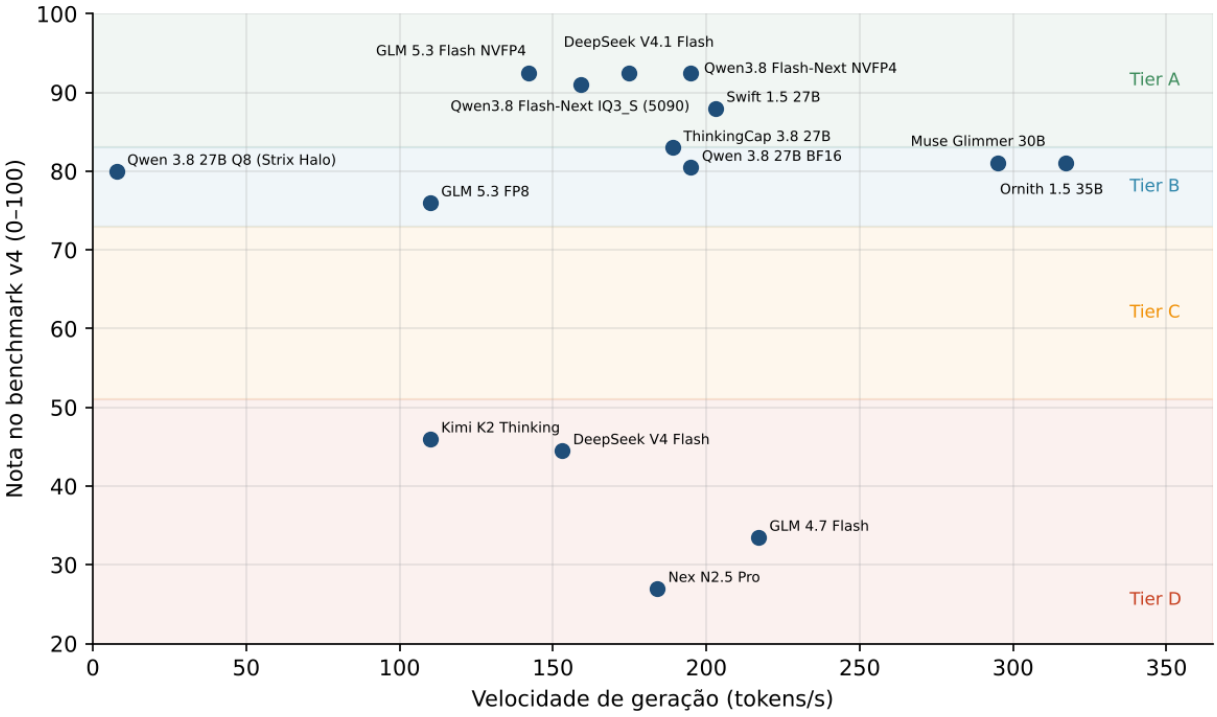
Vale ainda fazer fine-tuning? O vídeo Is Fine-Tuning Still Needed? LLMs, RAG, & LoRA lembra que o BloombergGPT perdeu para o GPT-4 e que modelos jurídicos especializados foram superados por generalistas. A ordem razoável é **prompt/contexto → RAG → skills → fine-tuning**, este só para gargalos específicos. Fine Tuning Large Language Models with InstructLab mostra o caminho com dados sintéticos e LoRA num modelo de 7B.

Quantização e serving: o benchmark v4

> **Para começar**
> Uma foto em JPEG ocupa bem menos que o original e quase ninguém nota. **Quantização** é guardar os pesos do modelo com menos precisão para caber em menos memória e rodar mais rápido. Já **serving** é como o modelo é posto no ar: o programa e as configurações.

Formato	Bits/peso	27B ocupa	Uso típico
BF16	16	~54 GB	Referência
FP8 / Q8_0	8	~27 GB	Produção; praticamente sem perda
NVFP4 / Q4_K_M	~4–4,5	~14–15 GB	GPUs recentes e llama.cpp
IQ3_S	~3,4	~11,5 GB	Modelos grandes em GPU de consumidor

Como o decode é limitado por banda, menos bits também significam mais tokens por segundo. Quanto de qualidade se perde? O post LLM Benchmark v4: o veredito de open source até out/2026 testa modelos abertos como agentes: construir um app Rails em 7 sprints enquanto um revisor injeta 14 defeitos de segurança e correção.



Nota no benchmark v4 versus velocidade de geração dos modelos abertos testados, com as faixas de tier. Velocidade e qualidade não se correlacionam. Fonte: blog AkitaOnRails (22/10/2026).

1. **Tamanho não garante qualidade.** O Kimi K2 Thinking, com ~1T de parâmetros, tirou 46; modelos de 27–35B ficaram entre 80 e 88. O que separa os grupos é o pós-treino para uso agêntico.
2. **Quantização pesa pouco.** Q8 tirou 80 contra 80,5 em BF16; um MoE em 3 bits numa GPU de consumidor perdeu só 1,5 ponto contra 4 bits num servidor, dentro do ruído.
3. **Serving pesa muito.** O mesmo checkpoint tirou 94 num provedor e 27 num servidor mal configurado. Template de chat errado, parser de chamadas de ferramenta quebrado ou contexto truncado destroem um bom modelo.

Velocidade depende do hardware, não da nota: o mesmo modelo de 27B gera 195 tokens por segundo num servidor de data center e menos de 8 numa máquina com memória unificada lenta, com a mesma pontuação.

> **Dica**
> Ao avaliar um modelo aberto, separe três variáveis: checkpoint, quantização e stack de serving (runtime, template, parser de ferramentas). Troque uma por vez: a terceira pode valer 60 pontos; a segunda raramente passa de 2.

Panorama de modelos (out/2026)

> **Para começar**
> Escolher um modelo é como escolher um veículo: não se usa caminhão para ir à padaria. Há modelos topo de linha, intermediários, leves para grande volume e locais.

O vídeo Every AI Model Explained In 20 Minutes (Update), de Tina Huang, organiza o mercado em camadas, de forma opinativa mas útil:

Camada	Exemplos	Quando usar
Flagship	Claude Opus 5/5.5, GPT-6 Sol, Gemini 3.1 Pro; aberto: Kimi K3	Problemas difíceis, código em projetos grandes
Intermediário	Claude Sonnet 5, Gemini 3.6 Flash; abertos: GLM 5.3, DeepSeek V4 Pro, Qwen 3	O dia a dia, agentes com custo controlado
Leve	Haiku 4.5, Gemini 3.5 Flash-Lite; abertos: DeepSeek V4 Flash, Mistral Small 4	Subagentes, classificação, volume
Local	Qwen, Gemma 4, Phi, GPT-OSS, Granite 4.0	Privacidade, baixa latência, custo zero por token

Uma regra prática da IBM: um modelo pequeno bem treinado entrega ~90% da qualidade a uma fração do custo em tarefas focadas. Modelos grandes seguem necessários para código amplo e documentos pesados.

Comparações diretas ajudam a calibrar a tabela. No vídeo sobre o GLM 5.3, ele empatou com o Claude em velocidade de agente numa caça a bugs, mas perdeu em calibração. Num teste de mapa interativo em HTML (Claude Has the One Thing ChatGPT Can't Copy. Here's What It Is.), vários modelos de fronteira entregaram resultados funcionais, e o autor conclui que a inteligência está empatada e o diferencial passou a ser “gosto” e ferramentas.

> **Atenção**
> Um vício comum em anúncios é comparar o modelo novo com concorrentes de meses atrás. Procure a data dos modelos em toda tabela de lançamento.

Inferência na prática: prompt caching e métricas

O **prompt caching** (What is Prompt Caching? Optimize LLM Latency with AI Transformers) guarda o KV cache de um prefixo do prompt para reutilizá-lo; não é cache de resposta. A correspondência é por prefixo: no primeiro token diferente, o cache acaba. Daí a regra: **estático primeiro, dinâmico por último** (sistema → documentos → exemplos → pergunta). A economia pode chegar a 90% do custo da entrada.

Para medir um sistema (LLM & AI Agent Benchmarks vs Reality: Why AI Applications Break), separe **TTFT** (tempo até o primeiro token, dominado pelo prefill), latência entre tokens (decode) e throughput, e teste com a forma de carga real: chat tem entrada curta e saída longa; RAG, o contrário. Técnicas de serving como batching contínuo, PagedAttention e *speculative decoding* (um modelo pequeno propõe tokens que o grande verifica em lote) completam o quadro. Defina metas de latência (por exemplo, p99 abaixo de 300 ms) e procure o ponto em que o TTFT dispara à medida que a concorrência cresce: é ali que o sistema precisa de mais hardware ou de outra configuração.

Limitações: alucinação, raciocínio infiel e contexto longo

> **Para começar**
> Um LLM é como um aluno muito lido que, na dúvida, prefere chutar com confiança a dizer "não sei". Ele pode inventar fatos, explicar um raciocínio diferente do que usou e se perder em textos longos.

Por que modelos alucinam

O modelo é treinado para produzir continuações plausíveis, e plausível não é verdadeiro. A pesquisa de **interpretabilidade** da Anthropic encontrou um mecanismo: um circuito decide "conheço esta entidade?" e, se ativo, inibe a resposta "não sei". A alucinação ocorre quando ele dispara por familiaridade sem que o fato esteja armazenado. Segundo 5 AI Myths & The Truth Behind Them: ML, Context, Agents & More, modelos de fronteira alucinam ~3% das vezes em benchmarks típicos, graças a busca, treino para recusa e raciocínio. Em fatos obscuros, números exatos e referências bibliográficas, a taxa é muito maior.

O raciocínio exibido não é o que o modelo fez

A cadeia de pensamento é texto gerado como qualquer outro. Em Interpretability: Understanding how AI models think, ao somar 36+59 o modelo explica o "vai um" escolar, mas internamente usa caminhos paralelos (estimativa da magnitude e cálculo do último dígito); recebendo a dica de uma resposta, trabalha de trás para frente para chegar a ela. A interpretabilidade também mostra o que acontece de fato:

- **Planejamento** (Tracing the thoughts of a large language model): ao escrever um verso que deve rimar com "grab it", o Claude ativa "rabbit" antes de escrever a linha.
- **Raciocínio em vários saltos**: Dallas → Texas → Austin; trocar internamente "Texas" por "Califórnia" produz Sacramento.
- **Passos não escritos** (The different levels of how Claude thinks): etapas intermediárias aparecem nas ativações sem ser escritas.
- **Consciência de teste** (Translating Claude's thoughts into language): num cenário de segurança, as ativações diziam "provavelmente é uma avaliação", o que expõe um limite dos próprios testes.

Os pesquisadores reconhecem o alcance limitado: essas ferramentas funcionam numa fração pequena dos casos.

A mesma pesquisa mostra que, em modelos grandes, conceitos como "oposto de grande" usam o mesmo circuito em vários idiomas, enquanto modelos pequenos mantêm circuitos separados por idioma. É uma pista de por que modelos maiores transferem melhor o que aprendem numa língua para outra.

Contexto longo e erro composto

Janelas de 1M de tokens acertam quase 100% em achar uma única “agulha no palheiro”, mas com várias agulhas caem 30–60 pontos entre 200 mil e 1 milhão de tokens. Use contexto longo para leitura e recuperação estruturada (RAG, busca, banco) para fatos exatos.

Um agente é um LLM num loop (objetivo → decidir → agir → observar). Se cada passo acerta 95%, a chance de 20 passos sem erro é ~36%; de 50 passos, ~8%. Por isso agentes em produção precisam de verificadores, humano no loop para ações irreversíveis e, como insiste o Akita, um “oráculo” (testes, documentação) que dê feedback real.

Em resumo

As limitações dos LLMs vêm da mesma origem do seu poder: continuações plausíveis, aprendidas de dados e moldadas por preferências. Alucinação, bajulação, raciocínio infiel e erro composto se tratam com sistemas que verificam: ferramentas, testes, calibração e engenharia tradicional.

Checklist do capítulo

- Conte tokens no tokenizer do modelo que vai usar; português custa mais que inglês.
- Saída é cara porque o decode é serial e limitado por banda.
- Em MoE, parâmetros totais definem a memória; ativos definem a velocidade.
- Q8 é praticamente sem perda; uma stack de serving errada pode custar dezenas de pontos.
- Reasoning effort mais alto não é sinônimo de melhor; meça.
- Prompt caching: estático primeiro, dinâmico por último.
- Antes de fine-tuning, esgote prompt, RAG e skills.
- Não trate a cadeia de raciocínio exibida como explicação; confie em testes.

Fontes deste capítulo

Título	URL	Canal / autor	Ponto principal
Interpretability: Understanding how AI models think	https://www.youtube.com/watch?v=fGKNUvivvnc	Anthropic	Circuitos de alucinação e raciocínio infiel
New frontier AI models, TypeSafe’s Jev AI, & NASA’s IBM collab	https://www.youtube.com/watch?v=O4n1jtWzt30	IBM Technology	Disputa vira “inteligência do sistema”
¿Es El Fin De Los LLM? (NCM)	https://www.youtube.com/watch?v=2BicjJgRkms	Jorge Marfil	NCM comparado a difusão; decode limitado por banda
LLM & AI Agent Benchmarks vs Reality: Why AI Applications Break	https://www.youtube.com/watch?v=nVImVgKpoOY	IBM Technology	TTFT, prefill/decode e formas de carga
Every AI Model Explained In 20 Minutes (Update)	https://www.youtube.com/watch?v=--8pJvYNcX4	Tina Huang	Camadas de modelos: flagship a local
GLM 5.3 é a IA Aberta que Encostou no Claude	https://www.youtube.com/watch?v=cEpynX9YsBw	ViktorKav	Pós-treino como alavanca; reasoning baixo basta
O que são tokens?	https://www.youtube.com/watch?v=tAhmBB_LSsc	Fernanda Kipper	Subpalavras e diferenças entre tokenizers

Título	URL	Canal / autor	Ponto principal
Se você só pode assistir UM vídeo sobre LLMs, assista esse.	https://www.youtube.com/watch?v=qaQn978_S40	Fernanda Kipper	Tokens, embeddings, posição e atenção
Claude Has the One Thing ChatGPT Can't Copy. Here's What It Is.	https://www.youtube.com/watch?v=We0XRlxxD4I	Corey McClain	Inteligência empatada; diferencial é gosto
What Is Jev? The AI Model That Doesn't Generate Text	https://www.youtube.com/watch?v=YGgNBcIgI4s	IBM Technology	Decisões calibradas sem gerar texto
5 AI Myths & The Truth Behind Them: ML, Context, Agents & More	https://www.youtube.com/watch?v=OWPRU_Pc4Ng	IBM Technology	Alucinação, inferência e erro composto
What are Word Embeddings?	https://www.youtube.com/watch?v=wgfSDrqYMJ4	IBM Technology	TF-IDF, Word2Vec, GloVe e contextuais
Fine Tuning Large Language Models with InstructLab	https://www.youtube.com/watch?v=pu3-PeBG0YU	IBM Technology	Dados sintéticos e LoRA num 7B
Granite 4.0: Small AI Models, Big Efficiency	https://www.youtube.com/watch?v=AaCBiGWTuyA	IBM Technology	Híbrido Mamba/Transformer, menos memória
What is DeepSeek? AI Model Basics Explained	https://www.youtube.com/watch?v=KTonvXhsxpc	IBM Technology	Linhagem DeepSeek: MoE e RL
Is Fine-Tuning Still Needed? LLMs, RAG, & LoRA	https://www.youtube.com/watch?v=-W2JdSI1v48	IBM Technology	Prompt → RAG → skills → fine-tuning
GPT-5: Five AI Model Improvements to Address LLM Weaknesses	https://www.youtube.com/watch?v=TY9CYRBOBPM	IBM Technology	Alucinação, bajulação e engano
What is AI Slop? Low-Quality AI Content Causes, Signs, & Fixes	https://www.youtube.com/watch?v=hI6mANth6oA	IBM Technology	Marcas e causas do texto genérico
AI Agents vs Mixture of Experts: AI Workflows Explained	https://www.youtube.com/watch?v=4-FH09AMsp0	IBM Technology	Agentes roteiam tarefas; MoE roteia tokens
What is Prompt Caching? Optimize LLM Latency with AI Transformers	https://www.youtube.com/watch?v=u57EnkQaUTY	IBM Technology	Cache de prefixo; estático primeiro
Transforming Language with Generative Pre-trained Transformers (GPT)	https://www.youtube.com/watch?v=bdICz_sBI34	IBM Technology	Sigla GPT e histórico
Small vs. Large AI Models: Trade-offs & Use Cases Explained	https://www.youtube.com/watch?v=0Wwn5IEqFcg	IBM Technology	Modelos pequenos alcançam os grandes
AI and Large Language Models Boost Language Translation	https://www.youtube.com/watch?v=TabyC8otFY8	IBM Technology	Tradução e arquitetura encoder-decoder
The different levels of how Claude thinks	https://www.youtube.com/watch?v=rKV5JcALQoQ	Anthropic	Passos internos não escritos
NCM matou o LLM: dessa vez, parece que é real	https://www.youtube.com/watch?v=PhTQOig_QA4	IA em 6 minutos	Onde o NCM ganha e perde
How Large Language Models Work	https://www.youtube.com/watch?v=5sLYAQS9sWQ	IBM Technology	Dados, arquitetura e treino
Translating Claude's thoughts into language	https://www.youtube.com/watch?v=j2knrqAzYVY	Anthropic	Modelo percebe estar num teste
Tracing the thoughts of a large language model	https://www.youtube.com/watch?v=Bj9BD2D3DzA	Anthropic	O modelo planeja a rima antes
LLMs + Agentes = competente o suficiente	https://akitaonrails.com/2026/10/12/llms-agentes-competente-o-suficiente/	Fabio Akita (AkitaOnRails)	Linha do tempo; ganho vem do harness
Treinei meu próprio modelo de IA	https://akitaonrails.com/2026/10/14/treinei-meu-proprio-modelo-de-ia-spoiler-e-so-pra-estudo/	Fabio Akita (AkitaOnRails)	Treino de 337M; zero em agentes
LLM Benchmark v4	https://akitaonrails.com/2026/10/22/llm-benchmark-v4-o-veredito-de-open-source-ate-out-2026/	Fabio Akita (AkitaOnRails)	Serving pesa mais que quantização
Melhor LLM Open Source: Strata com Qwen 3.8 Flash Next	https://akitaonrails.com/2026/10/22/melhor-llm-open-source-strata-com-qwen-3-8-flash-next/	Fabio Akita (AkitaOnRails)	VRAM como cache de especialistas
Entendendo o Hype do Jev: pra que serve?	https://akitaonrails.com/2026/10/03/entendendo-o-hype-do-jev-pra-que-serve/	Fabio Akita (AkitaOnRails)	Velocidade vem de trocar chat por classificador
Training Compute-Optimal Large Language Models (Chinchilla)	https://arxiv.org/abs/2203.15556	Hoffmann et al., DeepMind (2022)	~20 tokens por parâmetro
Scaling Laws for Neural Language Models	https://arxiv.org/abs/2001.08361	Kaplan et al., OpenAI (2020)	Perda cai como lei de potência

Título	URL	Canal / autor	Ponto principal
Outrageously Large Neural Networks: The Sparsely-Gated MoE Layer	https://arxiv.org/abs/1701.06538	Shazeer et al., Google (2017)	Camada MoE esparsa com top-k
Mixtral of Experts	https://arxiv.org/abs/2401.04088	Mistral AI (2024)	46,7B totais, 12,9B ativos
DeepSeek-V3 Technical Report	https://arxiv.org/abs/2412.19437	DeepSeek-AI (2024)	671B totais, 37B ativos
Qwen3 Technical Report	https://arxiv.org/abs/2505.09388	Equipe Qwen, Alibaba (2025)	Qwen3-235B-A22B e 30B-A3B
Analysis: OpenAI gpt-oss models	https://artificialanalysis.ai/articles/analysis-openai-gpt-oss-models	Artificial Analysis (2025)	Parâmetros totais e ativos do gpt-oss
Llama 4 Maverick 17B 128E Instruct	https://telnyx.com/llm-library/llama-4-maverick-17b-128e-instruct	Telnyx LLM Library	400B totais, 17B ativos
LoRA: Low-Rank Adaptation of Large Language Models	https://arxiv.org/abs/2106.09685	Hu et al., Microsoft (2021)	10.000× menos parâmetros treináveis
QLoRA: Efficient Finetuning of Quantized LLMs	https://arxiv.org/abs/2305.14314	Dettmers et al. (2023)	65B ajustado numa GPU de 48 GB
Adaptive Mixtures of Local Experts	https://www.cs.toronto.edu/~hinton/absps/jjnh91.pdf	Jacobs, Jordan, Nowlan, Hinton (1991)	Origem da ideia de MoE
llama.cpp	https://github.com/ggml-org/llama.cpp	ggml-org	Offload de especialistas e adapters LoRA